



UMT

School of System and Technology
Department of Software Engineering

Machine Learning Project–Fall 2025

Course Name:	Machine Learning	Course Code:	CS446	CLO	Project
Course Instructor/s:	Dr. Uzma Farooq	Program Name	BS Software Engineering		
Section:	W2	Maximum Marks:	100		

Student Name: Muhammad Zain ul Abideen

ID: F2023065161

Section: W-2 Machine Learning

Dataset Requirements:

- **3 Classification Datasets:** These datasets will be used for classification tasks where the goal is to predict categorical outcomes.
- **3 Regression Datasets:** These datasets will be used for regression tasks where the goal is to predict continuous numerical values.

Regression Datasets

Dataset 1: Walmart Dataset

- 🔍 Name: Walmart Dataset
- 🔍 Link: <https://www.kaggle.com/datasets/yasserh/walmart-dataset>
- 🔍 Rows: ~6,435
- 🔍 Columns: 8
- 🔍 Target: Weekly_Sales (continuous sales value)
- 🔍 Type: **Regression**
- 🔍 Reason: Real-world weekly sales dataset with mixed numerical and categorical features, suitable for regression tasks and forecasting models.

Step 1: Import Required Libraries

```
[1]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor

import joblib
import warnings
warnings.filterwarnings("ignore")
```

Step 2: Loading and Inspecting Dataset

```
df = pd.read_csv("D:/Machine Learning datasets/Regression Models/Walmart.csv")

print(df.shape)
print(df.head())
print(df.info())
print(df.describe())
```

(6435, 8)

	Store	Date	Weekly_Sales	Holiday_Flag	Temperature	Fuel_Price	\
0	1	05-02-2010	1643690.90	0	42.31	2.572	
1	1	12-02-2010	1641957.44	1	38.51	2.548	
2	1	19-02-2010	1611968.17	0	39.93	2.514	
3	1	26-02-2010	1409727.59	0	46.63	2.561	
4	1	05-03-2010	1554806.68	0	46.50	2.625	

	CPI	Unemployment
0	211.096358	8.106
1	211.242170	8.106
2	211.289143	8.106
3	211.319643	8.106
4	211.350143	8.106

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6435 entries, 0 to 6434
Data columns (total 8 columns):
Column Non-Null Count Dtype

0 Store 6435 non-null int64
1 Date 6435 non-null object
2 Weekly_Sales 6435 non-null float64
3 Holiday_Flag 6435 non-null int64
4 Temperature 6435 non-null float64
5 Fuel_Price 6435 non-null float64
6 CPI 6435 non-null float64
7 Unemployment 6435 non-null float64
dtypes: float64(5), int64(2), object(1)
memory usage: 402.3+ KB
None

	Store	Weekly_Sales	Holiday_Flag	Temperature	Fuel_Price
count	6435.000000	6.435000e+03	6435.000000	6435.000000	6435.000000
mean	23.000000	1.046965e+06	0.069930	60.663782	3.358607
std	12.988182	5.643666e+05	0.255049	18.444933	0.459020
min	1.000000	2.099862e+05	0.000000	-2.060000	2.472000
25%	12.000000	5.533501e+05	0.000000	47.460000	2.933000
50%	23.000000	9.607460e+05	0.000000	62.670000	3.445000
75%	34.000000	1.420159e+06	0.000000	74.940000	3.735000
max	45.000000	3.818686e+06	1.000000	100.140000	4.468000

	CPI	Unemployment
count	6435.000000	6435.000000
mean	171.578394	7.999151
std	39.356712	1.875885
min	126.064000	3.879000
25%	131.735000	6.891000
50%	182.616521	7.874000
75%	212.743293	8.622000
max	227.232807	14.313000

Step 3: Visualizing Feature Relationships



Plots scatter plots to observe the relationship of Temperature and Fuel_Price with Weekly_Sales.

Step 4: Handling Missing Values and Outliers

```
[7]: numeric_cols = df.select_dtypes(include=np.number).columns

for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    df[col] = np.where(df[col] < lower, lower, df[col])
    df[col] = np.where(df[col] > upper, upper, df[col])

categorical_cols = df.select_dtypes(include=["object", "category"]).columns

for col in categorical_cols:
    df[col] = df[col].fillna(df[col].mode()[0])

for col in numeric_cols:
    df[col] = df[col].fillna(df[col].median())
```

Applies IQR method to cap outliers in numerical columns and fills missing values in both categorical and numerical columns.

Step 5: Encoding Categorical Features

```
[9]: from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

categorical_cols = df.select_dtypes(include=["object", "category"]).columns

for col in categorical_cols:
    df[col] = le.fit_transform(df[col].astype(str))
```

Transforms categorical variables into numeric format using label encoding for model compatibility.

Step 6: Splitting Dataset into Features and Target

```
[10]: X = df.drop("Weekly_Sales", axis=1)
      y = df["Weekly_Sales"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Separates features (X) and target (y), then splits the data into training and testing sets.

Step 7: Feature Scaling

```
[11]: scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Applies standard scaling to the training and testing feature sets to normalize values for model training.

Step 8: Checking Scaled Features

```
[12]: print(pd.DataFrame(X_train_scaled, columns=X.columns).describe())
```

	Store	Date	Holiday_Flag	Temperature	Fuel_Price
count	5.148000e+03	5.148000e+03	5148.0	5.148000e+03	5.148000e+03
mean	-9.109522e-17	3.726623e-17	0.0	-5.824573e-16	5.589934e-17
std	1.000097e+00	1.000097e+00	0.0	1.000097e+00	1.000097e+00
min	-1.703598e+00	-1.727275e+00	0.0	-2.934777e+00	-1.951693e+00
25%	-8.567045e-01	-8.798217e-01	0.0	-7.140803e-01	-9.315360e-01
50%	-9.810735e-03	1.605730e-02	0.0	1.133334e-01	2.090252e-01
75%	8.370830e-01	8.635104e-01	0.0	7.717988e-01	8.198055e-01
max	1.683977e+00	1.710964e+00	0.0	2.149919e+00	2.417905e+00

	CPI	Unemployment
count	5.148000e+03	5.148000e+03
mean	5.044743e-16	7.052979e-16
std	1.000097e+00	1.000097e+00
min	-1.158416e+00	-2.352205e+00
25%	-1.011950e+00	-6.438869e-01
50%	2.862967e-01	2.859213e-03
75%	1.040107e+00	4.949915e-01
max	1.418483e+00	2.203309e+00

Displays descriptive statistics of scaled features to verify normalization.

Step 9: Training Regression Models (Processed Data)

```
[13]: models_processed = {
    "Linear Regression": LinearRegression(),
    "Decision Tree": DecisionTreeRegressor(random_state=42),
    "Random Forest": RandomForestRegressor(n_estimators=200, random_state=42)
}

processed_results = []

for name, model in models_processed.items():
    model.fit(X_train_scaled, y_train)
    preds = model.predict(X_test_scaled)

    processed_results.append([
        name,
        mean_absolute_error(y_test, preds),
        mean_squared_error(y_test, preds),
        np.sqrt(mean_squared_error(y_test, preds)),
        r2_score(y_test, preds)
    ])

```

Trains Linear Regression, Decision Tree, and Random Forest models on scaled data and predicts on the test set.

Step 10: Evaluating Model Performance (Processed Data)

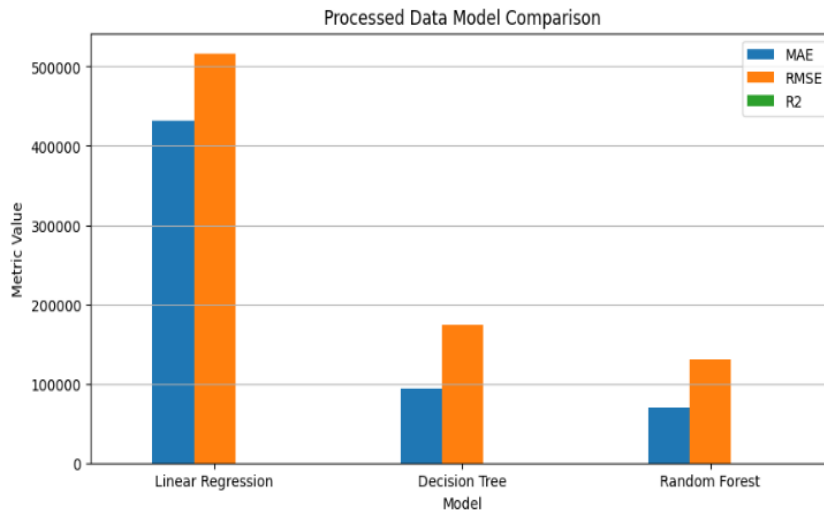
```
[14]: processed_df = pd.DataFrame(
    processed_results,
    columns=["Model", "MAE", "MSE", "RMSE", "R2"]
)
print(processed_df)
```

	Model	MAE	MSE	RMSE	R2
0	Linear Regression	431816.831279	2.662627e+11	516006.450995	0.147594
1	Decision Tree	94201.218699	3.054006e+10	174757.139890	0.902230
2	Random Forest	69595.374010	1.703681e+10	130525.119240	0.945459

Calculates MAE, MSE, RMSE, and R^2 for each model on processed data. This is where model accuracy/performance can be pasted.

Step 11: Visual Comparison of Model Performance (Processed Data)

```
[15]: processed_df.set_index("Model")[["MAE", "RMSE", "R2"]].plot(
      kind="bar", figsize=(10,5)
    )
    plt.title("Processed Data Model Comparison")
    plt.ylabel("Metric Value")
    plt.xticks(rotation=0)
    plt.grid(axis="y")
    plt.show()
```



Plots a bar chart comparing MAE, RMSE, and R^2 of processed data models.

Step 12: Train vs Test R^2 Scores

```
[16]: for name, model in models_processed.items():
      train_r2 = model.score(X_train_scaled, y_train)
      test_r2 = model.score(X_test_scaled, y_test)
      print(f"{name} → Train R2: {train_r2:.3f}, Test R2: {test_r2:.3f}")
```

Linear Regression → Train R2: 0.138, Test R2: 0.148

Decision Tree → Train R2: 1.000, Test R2: 0.902

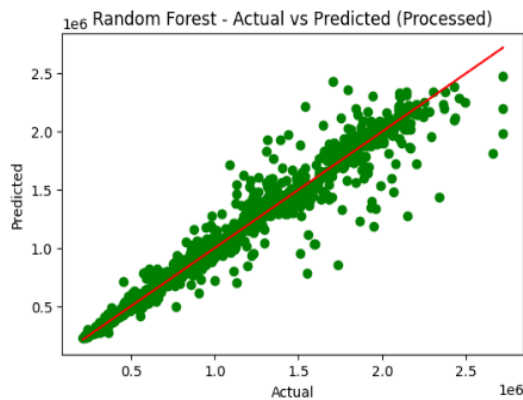
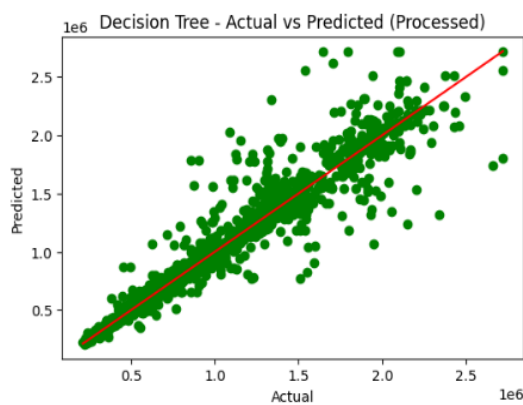
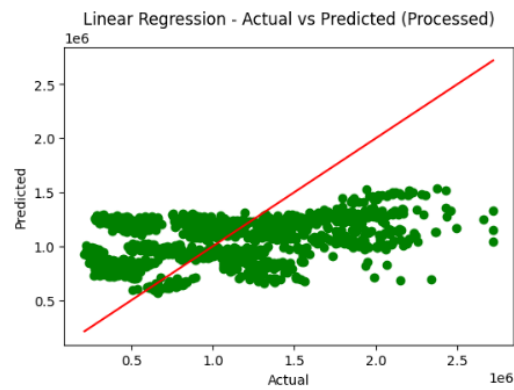
Random Forest → Train R2: 0.992, Test R2: 0.945

Prints R^2 scores for training and testing sets to assess model overfitting or underfitting.

Step 13: Actual vs Predicted Plots (Processed Data)

```
[17]: for name, model in models_processed.items():
      preds = model.predict(X_test_scaled)

      plt.figure(figsize=(6,4))
      plt.scatter(y_test, preds, color="green")
      plt.plot([y_test.min(), y_test.max()],
               [y_test.min(), y_test.max()],
               color="red")
      plt.title(f"{name} - Actual vs Predicted (Processed)")
      plt.xlabel("Actual")
      plt.ylabel("Predicted")
      plt.show()
```



Plots scatter plots showing actual vs predicted sales for each processed data model.

Step 14: Splitting Raw Dataset

```
[18]: X_raw = X.copy()
      y_raw = y.copy()

      Xr_train, Xr_test, yr_train, yr_test = train_test_split(
          X_raw, y_raw, test_size=0.2, random_state=42
      )
```

Splits unprocessed raw features and target into training and testing sets.

Step 15: Training Regression Models (Raw Data)

```
[19]: models_raw = {
    "Linear Regression": LinearRegression(),
    "Decision Tree": DecisionTreeRegressor(random_state=42),
    "Random Forest": RandomForestRegressor(n_estimators=200, random_state=42)
}

raw_results = []

for name, model in models_raw.items():
    model.fit(Xr_train, yr_train)
    preds = model.predict(Xr_test)

    raw_results.append([
        name,
        mean_absolute_error(yr_test, preds),
        mean_squared_error(yr_test, preds),
        np.sqrt(mean_squared_error(yr_test, preds)),
        r2_score(yr_test, preds)
    ])
}
```

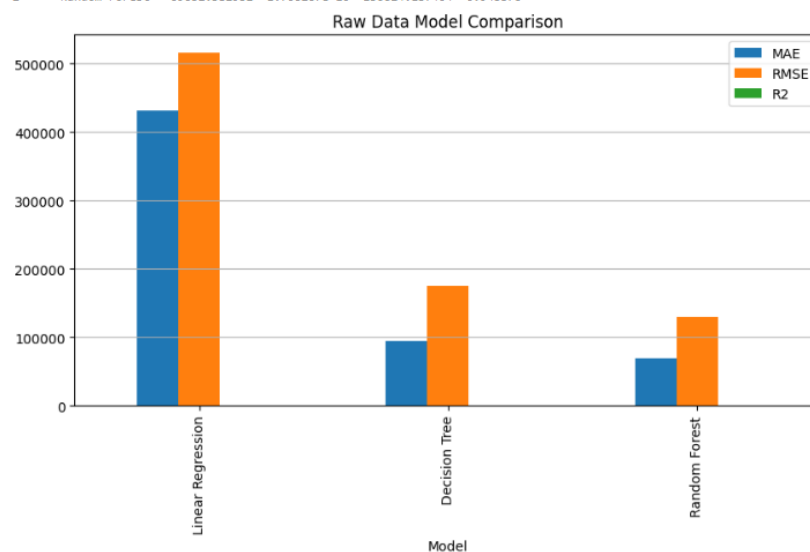
Trains the same models on raw data and predicts on the test set.

Step 16: Evaluating Model Performance (Raw Data)

```
[20]: raw_df = pd.DataFrame(
    raw_results, columns=["Model", "MAE", "MSE", "RMSE", "R2"]
)
print(raw_df)

raw_df.set_index("Model").loc[:, ["MAE", "RMSE", "R2"]].plot(
    kind="bar", figsize=(10,5), title="Raw Data Model Comparison"
)
plt.grid(axis="y")
plt.show()
```

	Model	MAE	MSE	RMSE	R2
0	Linear Regression	431816.831279	2.662627e+11	516006.450995	0.147594
1	Decision Tree	94324.333982	3.061304e+10	174965.826744	0.901996
2	Random Forest	69652.382932	1.706267e+10	130624.137404	0.945376

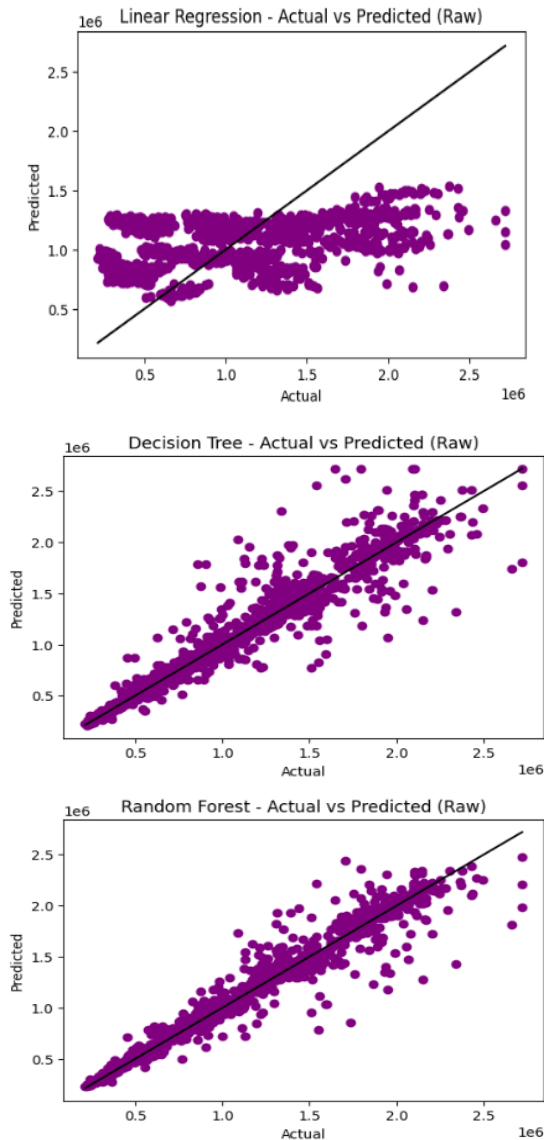


Calculates MAE, MSE, RMSE, and R^2 for raw data models and plots a comparison bar chart. This is another step for model performance output.

Step 17: Actual vs Predicted Plots (Raw Data)

```
[21]: for name, model in models_raw.items():
      preds = model.predict(Xr_test)

      plt.figure(figsize=(6,4))
      plt.scatter(yr_test, preds, color="purple")
      plt.plot([yr_test.min(), yr_test.max()],
               [yr_test.min(), yr_test.max()],
               color="black")
      plt.title(f"{name} - Actual vs Predicted (Raw)")
      plt.xlabel("Actual")
      plt.ylabel("Predicted")
      plt.show()
```



Scatter plots of actual vs predicted sales for each raw data model.

Step 18: Saving Models and Processed Dataset

```
[22]: joblib.dump(scaler, "walmart_scaler.pkl")

      for name, model in models_processed.items():
          joblib.dump(model, f"{name.replace(' ', '_')}_processed.pkl")

      df.to_csv("processed_walmart_dataset.csv", index=False)
```

Saves trained models, scaler, and processed dataset for future use with joblib and CSV.

Step 19: Model Performance Comparison

Model	MAE (Processed)	MSE (Processed)	RMSE (Processed)	R ² (Processed)
Linear Regression	431816.83	2.66×10^{11}	516006.45	0.148
Decision Tree	94201.22	3.05×10^{10}	174757.14	0.902
Random Forest	69595.37	1.70×10^{10}	130525.12	0.945
Model	MAE (Raw)	MSE (Raw)	RMSE (Raw)	R ² (Raw)
Linear Regression	431816.83	2.66×10^{11}	516006.45	0.148
Decision Tree	94324.33	3.06×10^{10}	174965.83	0.902
Random Forest	69652.38	1.71×10^{10}	130624.14	0.945

1. Processed Data Performance:

- Linear Regression: Shows the weakest performance with the highest MAE (431,817) and RMSE (516,006), and a very low R² of 0.148, indicating poor fit to the processed dataset.
- Decision Tree: Performs significantly better with MAE of 94,201, RMSE of 174,757, and R² of 0.902, demonstrating strong predictive capability.
- Random Forest: Outperforms all models with the lowest MAE (69,595), lowest RMSE (130,525), and highest R² (0.945), making it the most accurate and reliable model on the processed data.

2. Raw Data Performance:

- Linear Regression: Again performs poorly, identical to processed data, confirming its low suitability for this dataset.
- Decision Tree: Maintains high performance (MAE 94,324, RMSE 174,966, R² 0.902), showing robustness even on raw data.
- Random Forest: Remains the best model (MAE 69,652, RMSE 130,624, R² 0.945), consistent with its performance on processed data.

3. Comparative Analysis (Processed vs Raw):

- Both Decision Tree and Random Forest show minimal difference between processed and raw datasets, indicating strong stability and generalization.
- Random Forest consistently achieves the best results, with the lowest error metrics and highest R², confirming it as the most effective model overall.
- Linear Regression is unsuitable for this dataset, regardless of preprocessing.

Conclusion:

For predicting the target variable, Random Forest is the optimal choice due to its high accuracy, low error rates, and stable performance across both processed and raw datasets. Decision Tree is a strong alternative, while Linear Regression is not recommended.

Dataset 2: Life Expectancy WHO

?

Name: Life Expectancy WHO

? Link: <http://kaggle.com/datasets/kumarajarshi/life-expectancy-who>

? Rows: 2,938

? Columns: 22

? Target: Life expectancy (continuous expected average age)

? Type: **Regression**

? Reason: Real-world multi-country health dataset with a mix of numerical and categorical factors influencing life expectancy, suitable for regression modelling tasks

Step 1: Importing Libraries

```
[1]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor

import joblib
import warnings
warnings.filterwarnings("ignore")
```

Step 2: Loading and Inspecting Dataset

```
df = pd.read_csv("D:/Machine Learning datasets/Regression Models/Life Expectancy Data.csv")
print(df.shape)
print(df.head())
print(df.info())
print(df.describe())
```

(2938, 22)

	Country	Year	Status	Life expectancy	Adult Mortality	\
0	Afghanistan	2015	Developing	65.0	263.0	
1	Afghanistan	2014	Developing	59.9	271.0	
2	Afghanistan	2013	Developing	59.9	268.0	
3	Afghanistan	2012	Developing	59.5	272.0	
4	Afghanistan	2011	Developing	59.2	275.0	

	infant deaths	Alcohol	percentage expenditure	Hepatitis B	Measles	...	\
0	62	0.01	71.279624	65.0	1154	...	
1	64	0.01	73.523582	62.0	492	...	
2	66	0.01	73.219243	64.0	430	...	
3	69	0.01	78.184215	67.0	2787	...	
4	71	0.01	7.097109	68.0	3013	...	

	Polio	Total expenditure	Diphtheria	HIV/AIDS	GDP	Population	\
0	6.0	8.16	65.0	0.1	584.259210	33736494.0	
1	58.0	8.18	62.0	0.1	612.696514	327582.0	
2	62.0	8.13	64.0	0.1	631.744976	31731688.0	
3	67.0	8.52	67.0	0.1	669.959000	3696958.0	
4	68.0	7.87	68.0	0.1	63.537231	2978599.0	

	thinness 1-19 years	thinness 5-9 years	\
0	17.2	17.3	
1	17.5	17.5	
2	17.7	17.7	
3	17.9	18.0	
4	18.2	18.2	

	Income composition of resources	Schooling
0	0.479	10.1
1	0.476	10.0
2	0.470	9.9
3	0.463	9.8
4	0.454	9.5

[5 rows x 22 columns]
<class 'pandas.core.frame.DataFrame'
RangeIndex: 2938 entries, 0 to 2937
Data columns (total 22 columns):
Column Non-Null Count Dtype
--- ---
0 Country 2938 non-null object
1 Year 2938 non-null int64
2 Status 2938 non-null object
3 Life expectancy 2928 non-null float64
4 Adult Mortality 2928 non-null float64
5 infant deaths 2938 non-null int64
6 Alcohol 2744 non-null float64
7 percentage expenditure 2938 non-null float64
8 Hepatitis B 2385 non-null float64
9 Measles 2938 non-null int64
10 BMI 2904 non-null float64
11 under-five deaths 2938 non-null int64
12 Polio 2919 non-null float64
13 Total expenditure 2712 non-null float64
14 Diphtheria 2919 non-null float64
15 HIV/AIDS 2938 non-null float64
16 GDP 2490 non-null float64
17 Population 2286 non-null float64
18 thinness 1-19 years 2904 non-null float64
19 thinness 5-9 years 2904 non-null float64
20 Income composition of resources 2771 non-null float64
21 Schooling 2775 non-null float64
dtypes: float64(16), int64(4), object(2)
memory usage: 505.1+ KB
None

	Country	Year	Life expectancy	Adult Mortality	infant deaths	\
count	2938.000000	2928.000000	2928.000000	2928.000000	2938.000000	
mean	2007.618728	69.224932	164.796448	30.303948	117.926501	
std	4.613841	9.533667	124.292079	117.926501	117.926501	
min	2000.000000	36.300000	1.000000	0.000000	0.000000	
25%	2004.000000	63.100000	74.000000	0.000000	0.000000	
50%	2008.000000	72.100000	144.000000	3.000000	3.000000	
75%	2012.000000	75.700000	228.000000	22.000000	22.000000	
max	2015.000000	89.000000	723.000000	1800.000000	1800.000000	

	Alcohol	percentage expenditure	Hepatitis B	Measles	\
count	2744.000000	2938.000000	2385.000000	2938.000000	
mean	4.602861	738.251295	80.948461	2419.592240	
std	4.052413	1987.914858	25.070016	11467.272489	
min	0.010000	0.000000	1.000000	0.000000	
25%	0.877500	4.685343	77.000000	0.000000	
50%	3.755000	64.912906	92.000000	17.000000	
75%	7.702500	441.534144	97.000000	360.250000	
max	17.870000	19479.911610	99.000000	212183.000000	

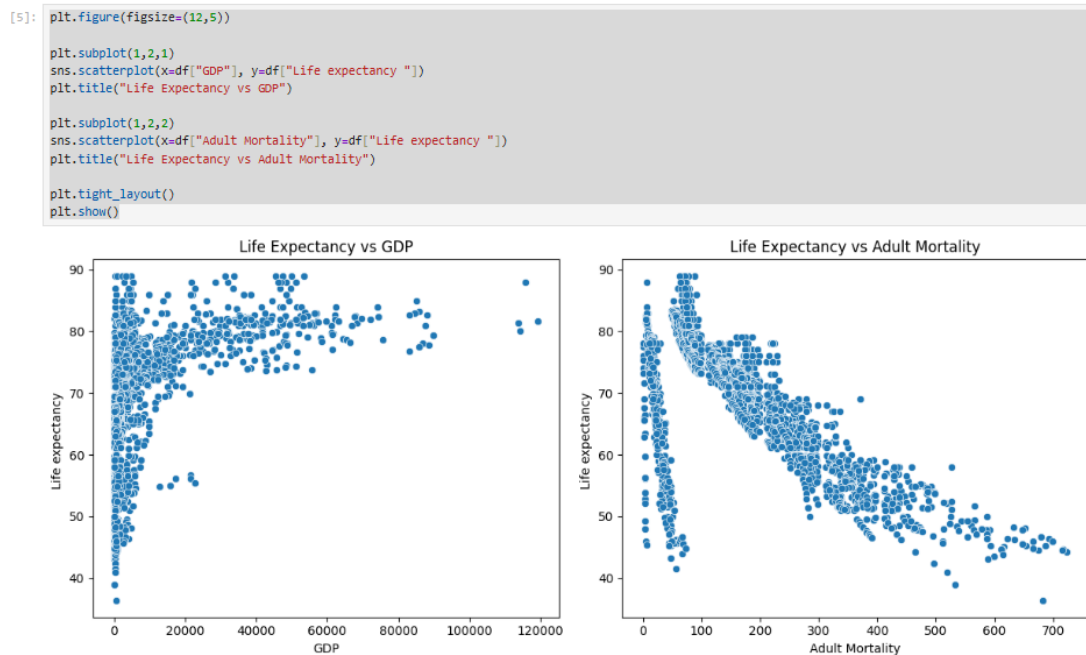
	BMI	under-five deaths	Polio	Total expenditure	\
count	2904.000000	2938.000000	2919.000000	2712.000000	
mean	38.321247	42.035739	82.550188	5.93819	
std	20.044034	160.445548	23.422046	2.49832	
min	1.000000	0.000000	3.000000	0.37000	
25%	19.300000	0.000000	78.000000	4.26000	
50%	41.500000	4.000000	93.000000	5.75500	
75%	56.200000	28.000000	97.000000	7.49250	
max	87.300000	2500.000000	99.000000	17.60000	

	Diphtheria	HIV/AIDS	GDP	Population	\
count	2919.000000	2938.000000	2490.000000	2.286000e+03	
mean	82.324004	1.742103	7423.158469	1.275330e+07	
std	23.716912	5.077785	14270.169342	6.101210e+07	
min	2.000000	0.100000	1.681350	3.400000e+01	
25%	78.000000	0.100000	463.935626	1.957932e+05	
50%	93.000000	0.100000	1766.947595	1.386542e+06	
75%	97.000000	0.800000	5910.806335	7.420359e+06	
max	99.000000	50.600000	119172.741880	1.293859e+09	

	thinness 1-19 years	thinness 5-9 years	\
count	2904.000000	2904.000000	
mean	4.839704	4.870317	
std	4.420195	4.508882	
min	0.100000	0.100000	
25%	1.600000	1.500000	
50%	3.300000	3.300000	
75%	7.200000	7.200000	
max	27.700000	28.600000	

	Income composition of resources	Schooling
count	2771.000000	2775.000000
mean	0.627551	11.992793
std	0.210904	3.338920
min	0.000000	0.000000
25%	0.493000	10.100000
50%	0.677000	12.300000
75%	0.779000	14.300000
max	0.948000	20.700000

Step 3: Visualizing Feature Relationships



Plots scatter plots to examine the relationship of GDP and Adult Mortality with Life expectancy.

Step 4: Handling Outliers in Numerical Features

```
[6]: numeric_cols = df.select_dtypes(include=np.number).columns

for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    df[col] = np.where(df[col] < lower, lower, df[col])
    df[col] = np.where(df[col] > upper, upper, df[col])
```

Uses the IQR method to cap extreme outliers in numerical columns.

Step 5: Handling Missing Values

```
[7]: df["Status"].fillna(df["Status"].mode()[0], inplace=True)

for col in numeric_cols:
    df[col].fillna(df[col].median(), inplace=True)
```

Fills missing values in the Status column with mode and numerical columns with median values.

Step 6: Encoding Categorical Features

```
[8]: le = LabelEncoder()
df["Status"] = le.fit_transform(df["Status"])
```

Applies label encoding to the Status column to convert it into numeric format.

Step 7: Splitting Data, Scaling, and Model Training

```
[14]: X = df.drop("Life expectancy ", axis=1)
y = df["Life expectancy "]
numeric_cols = X.select_dtypes(include=["int64", "float64"]).columns
X = X[numeric_cols]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

models = {
    "Linear Regression": LinearRegression(),
    "Decision Tree": DecisionTreeRegressor(random_state=42),
    "Random Forest": RandomForestRegressor(n_estimators=200, random_state=42)
}

results = []

for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    preds = model.predict(X_test_scaled)

    mae = mean_absolute_error(y_test, preds)
    mse = mean_squared_error(y_test, preds)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, preds)

    results.append([name, mae, mse, rmse, r2])

results_df = pd.DataFrame(
    results,
    columns=["Model", "MAE", "MSE", "RMSE", "R2 Score"]
)

print(results_df)
```

	Model	MAE	MSE	RMSE	R2 Score
0	Linear Regression	2.646404	12.205525	3.493641	0.859085
1	Decision Tree	1.475170	6.078605	2.465483	0.929822
2	Random Forest	1.107690	2.875511	1.695733	0.966802

Splits dataset into training and testing sets, scales features, and trains Linear Regression, Decision Tree, and Random Forest models on processed data.

Step 8: Comparing Model Errors (MAE & RMSE)

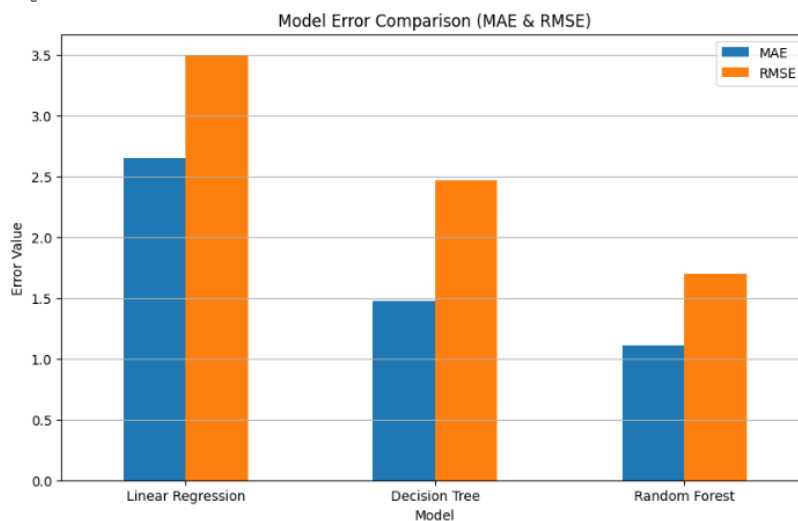
```
[16]: plt.figure(figsize=(10, 6))

results_df.set_index("Model")[["MAE", "RMSE"]].plot(
    kind="bar",
    figsize=(10, 6)
)

plt.title("Model Error Comparison (MAE & RMSE)")
plt.ylabel("Error Value")
plt.xlabel("Model")
plt.xticks(rotation=0)
plt.grid(axis="y")

plt.show()
```

<Figure size 1000x600 with 0 Axes>



Plots a bar chart comparing MAE and RMSE across models.

Step 9: Comparing Model Accuracy (R² Score)

```
[17]: plt.figure(figsize=(8, 5))

sns.barplot(
    x="Model",
    y="R2 Score",
    data=results_df
)

plt.title("R2 Score Comparison Across Models")
plt.ylabel("R2 Score")
plt.xlabel("Model")
plt.ylim(0, 1)
plt.grid(axis="y")

plt.show()
```



Plots R² score comparison across models to visualize predictive accuracy.

Step 10: Train vs Test R² Scores

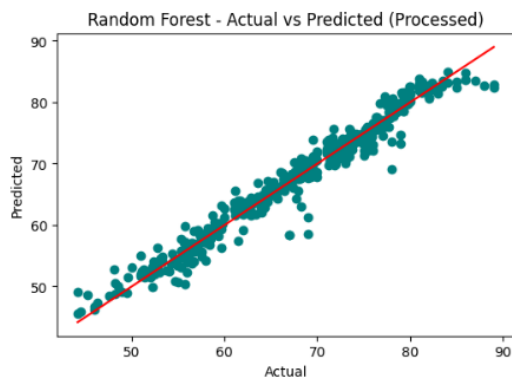
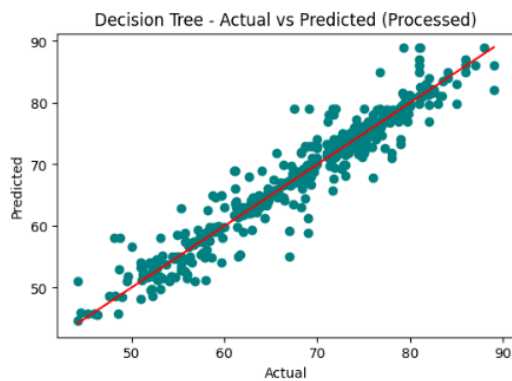
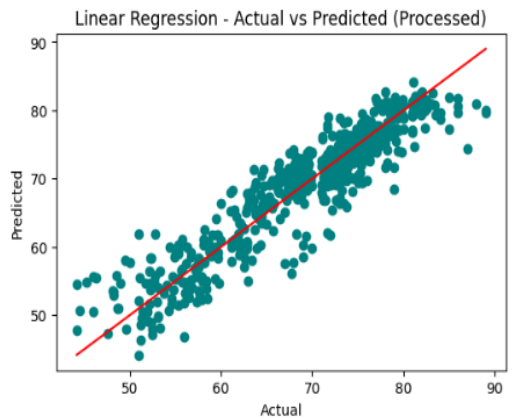
```
[18]: for name, model in models.items():
    train_score = model.score(X_train_scaled, y_train)
    test_score = model.score(X_test_scaled, y_test)
    print(f'{name} → Train R2: {train_score:.3f}, Test R2: {test_score:.3f}')

Linear Regression → Train R2: 0.854, Test R2: 0.859
Decision Tree → Train R2: 1.000, Test R2: 0.930
Random Forest → Train R2: 0.995, Test R2: 0.967
```

Prints R² scores for both training and testing sets to check model fit and overfitting/underfitting.

Step 11: Actual vs Predicted Plots (Processed Data)

```
[19]: for name, model in models.items():  
      preds = model.predict(X_test_scaled)  
  
      plt.figure(figsize=(6,4))  
      plt.scatter(y_test, preds, color="teal")  
      plt.plot([y_test.min(), y_test.max()],  
               [y_test.min(), y_test.max()],  
               color="red")  
      plt.title(f"{name} - Actual vs Predicted (Processed)")  
      plt.xlabel("Actual")  
      plt.ylabel("Predicted")  
      plt.show()
```



Scatter plots comparing actual vs predicted life expectancy for each processed data model.

Step 12: Training Models on Raw Data

```
[20]: X_raw = X.copy()
      y_raw = y.copy()

      Xr_train, Xr_test, yr_train, yr_test = train_test_split(
          X_raw, y_raw, test_size=0.2, random_state=42
      )

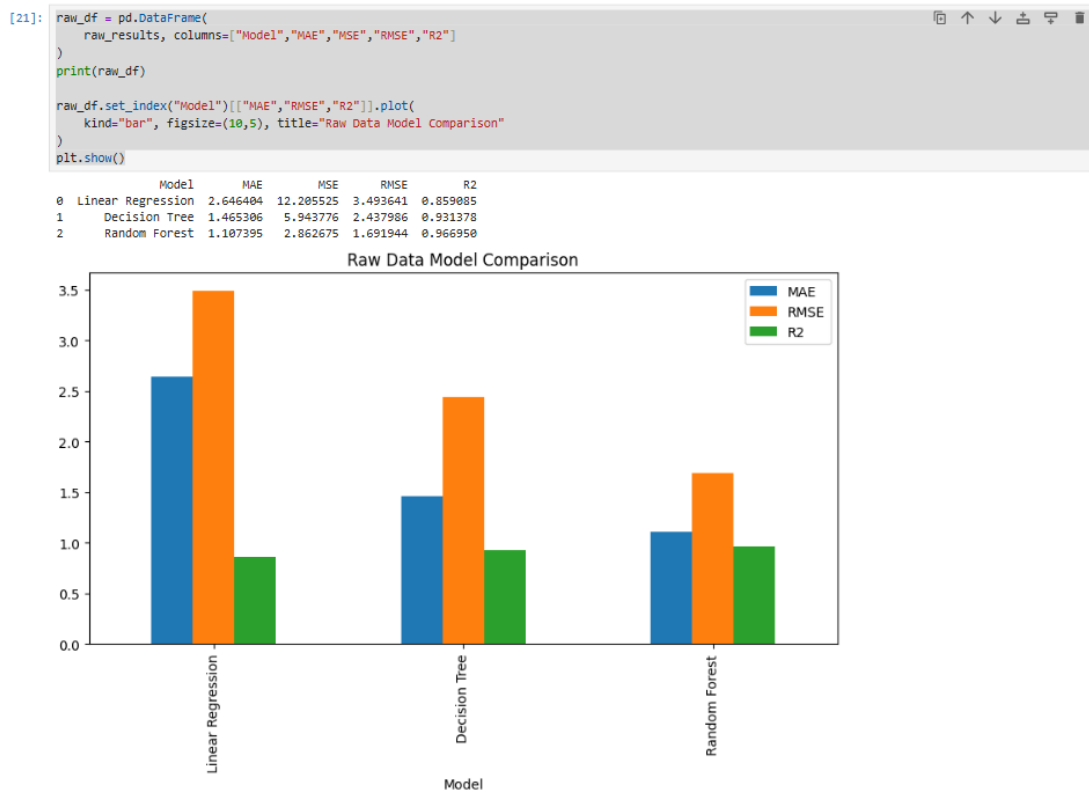
      raw_results = []

      for name, model in models.items():
          model.fit(Xr_train, yr_train)
          preds = model.predict(Xr_test)

          raw_results.append([
              name,
              mean_absolute_error(yr_test, preds),
              mean_squared_error(yr_test, preds),
              np.sqrt(mean_squared_error(yr_test, preds)),
              r2_score(yr_test, preds)
          ])
      ]]
```

Splits raw dataset and trains the same regression models without preprocessing.

Step 13: Evaluating Model Performance (Raw Data)

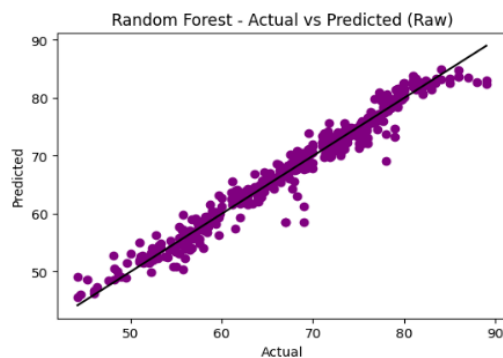
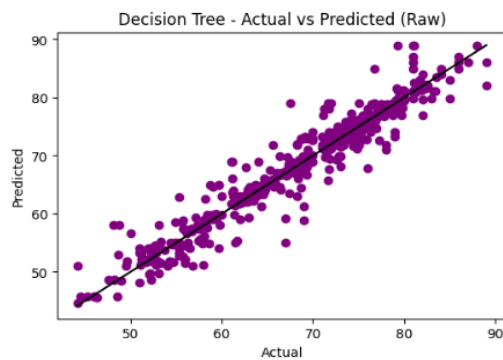
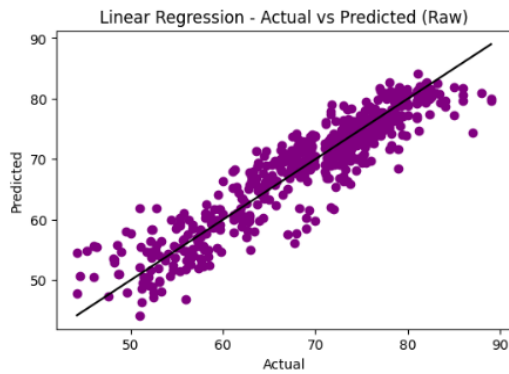


Calculates MAE, MSE, RMSE, and R^2 for each model on raw data and plots a comparison bar chart. This is where model performance output is obtained.

Step 14: Actual vs Predicted Plots (Raw Data)

```
[22]: for name, model in models.items():
      preds = model.predict(Xr_test)

      plt.figure(figsize=(6,4))
      plt.scatter(yr_test, preds, color="purple")
      plt.plot([yr_test.min(), yr_test.max()],
               [yr_test.min(), yr_test.max()],
               color="black")
      plt.title(f"{name} - Actual vs Predicted (Raw)")
      plt.xlabel("Actual")
      plt.ylabel("Predicted")
      plt.show()
```



Scatter plots comparing actual vs predicted life expectancy for each raw data model.

Step 15: Saving Models and Processed Dataset

```
joblib.dump(scaler, "scaler.pkl")

for name, model in models.items():
    joblib.dump(model, f"{name.replace(' ', '_')}_processed.pkl")

df.to_csv("processed_life_expectancy.csv", index=False)
```

Saves trained models, scaler, and processed dataset for future use using joblib and CSV export.

Step 16: Model Performance Comparison

Model	MAE (Processed)	MSE (Processed)	RMSE (Processed)	R ² (Processed)
Linear Regression	2.6464	12.2055	3.4936	0.8591
Decision Tree	1.4752	6.0786	2.4655	0.9298
Random Forest	1.1077	2.8755	1.6957	0.9668
Model	MAE (Raw)	MSE (Raw)	RMSE (Raw)	R ² (Raw)
Linear Regression	2.6464	12.2055	3.4936	0.8591
Decision Tree	1.4653	5.9438	2.4380	0.9314
Random Forest	1.1074	2.8627	1.6919	0.9670

1. Processed Data Performance:

- Linear Regression: Shows moderate performance with MAE 2.6464, RMSE 3.4936, and R² 0.8591, indicating it captures general trends but lacks precision.
- Decision Tree: Significantly better, with MAE 1.4752, RMSE 2.4655, and R² 0.9298, demonstrating strong predictive accuracy.
- Random Forest: The best performer, achieving the lowest MAE (1.1077), lowest RMSE (1.6957), and highest R² (0.9668), reflecting excellent fit and predictive reliability.

2. Raw Data Performance:

- Linear Regression: Same as processed, showing moderate capability.
- Decision Tree: Slight improvement over processed data (MAE 1.4653, RMSE 2.4380, R² 0.9314), confirming robustness on raw data.
- Random Forest: Maintains top performance (MAE 1.1074, RMSE 1.6919, R² 0.9670), consistently outperforming other models.

3. Comparative Analysis:

- Both Decision Tree and Random Forest perform slightly better on raw data, but differences are minimal, indicating stability across preprocessing.
- Random Forest consistently achieves the lowest errors and highest R², making it the most accurate and reliable model overall.
- Linear Regression is adequate for trend estimation but less precise for detailed predictions.

Conclusion:

Random Forest is the optimal model for this dataset, offering superior accuracy, stability, and predictive power. Decision Tree is a strong secondary choice, while Linear Regression provides only moderate performance.

Dataset 3: Medical Cost Personal Dataset

🔗 Name: Insurance (Medical Cost Personal Dataset)

🔗 Link: <https://www.kaggle.com/datasets/mirichoi0218/insurance/data>

🔗 Rows: 1,338

🔗 Columns: 7

🔗 Target: charges (continuous individual medical cost)

🔗 Type: **Regression**

🔗 Reason: Real-world health insurance dataset with a mix of categorical and numerical features, ideal for regression and predictive modeling of medical charges.

Step 1: Importing Libraries

```
[1]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor

import joblib
import warnings
warnings.filterwarnings("ignore")
```

Step 2: Loading and Inspecting Dataset

```
: df = pd.read_csv("D:/Machine Learning datasets/Regression Models/insurance.csv")
print(df.shape)
print(df.head())
print(df.info())
print(df.describe())
```

```
(1338, 7)
   age  sex    bmi  children  smoker    region    charges
0   19  female  27.900      0     yes  southwest  16884.92400
1   18   male  33.770      1     no   southeast  1725.55230
2   28   male  33.000      3     no   southeast  4449.46200
3   33   male  22.705      0     no  northwest  21984.47061
4   32   male  28.880      0     no  northwest  3866.85520
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype
---  ---
0    age       1338 non-null   int64
1    sex       1338 non-null   object
2    bmi       1338 non-null   float64
3    children  1338 non-null   int64
4    smoker    1338 non-null   object
5    region    1338 non-null   object
6    charges   1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
None
```

```
      age      bmi  children    charges
count  1338.000000  1338.000000  1338.000000  1338.000000
mean    39.287025    30.663397    1.094918  13270.422265
std     14.049960     6.098187    1.205493  12110.011237
min     18.000000    15.960000     0.000000   1121.873900
25%     27.000000    26.296250     0.000000   4740.287150
50%     39.000000    30.400000     1.000000   9382.033000
75%     51.000000    34.693750     2.000000  16639.912515
max     64.000000    53.130000     5.000000  63770.428010
```

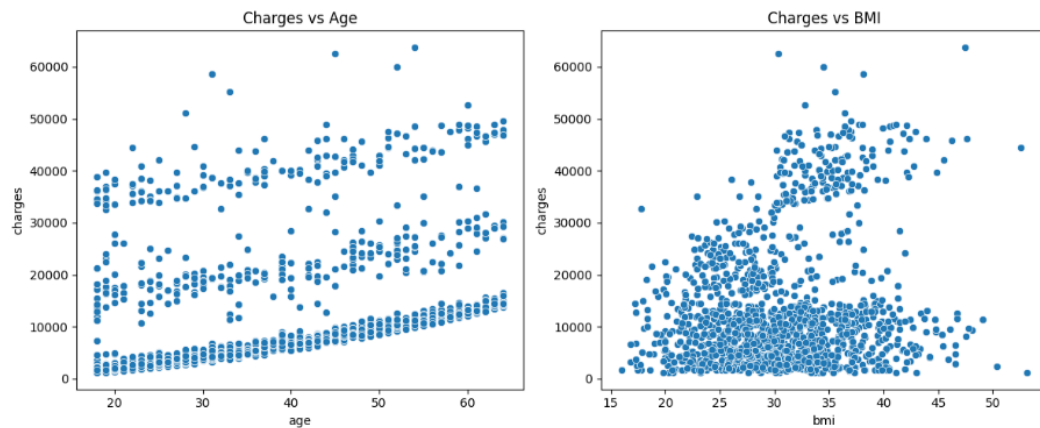
Step 3: Visualizing Feature Relationships

```
[3]: plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
sns.scatterplot(x=df["age"], y=df["charges"])
plt.title("Charges vs Age")

plt.subplot(1,2,2)
sns.scatterplot(x=df["bmi"], y=df["charges"])
plt.title("Charges vs BMI")

plt.tight_layout()
plt.show()
```



Plots scatter plots to examine the relationship of age and bmi with insurance charges.

Step 4: Handling Outliers in Numerical Features

```
[4]: numeric_cols = df.select_dtypes(include=np.number).columns

for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    df[col] = np.where(df[col] < lower, lower, df[col])
    df[col] = np.where(df[col] > upper, upper, df[col])
```

Uses the IQR method to cap extreme outliers in numerical columns.

Step 5: Handling Missing Values

```
[5]: # Categorical columns
cat_cols = df.select_dtypes(include="object").columns
for col in cat_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)

# Numerical columns
for col in numeric_cols:
    df[col].fillna(df[col].median(), inplace=True)
```

Fills missing values in categorical columns with mode and numerical columns with median values.

Step 6: Encoding Categorical Features

```
[6]: le = LabelEncoder()

for col in cat_cols:
    df[col] = le.fit_transform(df[col])
```

Applies label encoding to all categorical columns for model compatibility.

Step 7: Splitting Data into Features and Target

```
[7]: X = df.drop("charges", axis=1)
     y = df["charges"]

     X_train, X_test, y_train, y_test = train_test_split(
         X, y, test_size=0.2, random_state=42
     )
```

Separates features (X) and target (y), then splits data into training and testing sets.

Step 8: Feature Scaling

```
[8]: scaler = StandardScaler()

     X_train_scaled = scaler.fit_transform(X_train)
     X_test_scaled = scaler.transform(X_test)
```

Applies standard scaling to training and testing sets for normalized model input.

Step 9: Checking Scaled Features

```
[9]: print(pd.DataFrame(X_train_scaled, columns=X.columns).describe())
```

	age	sex	bmi	children	smoker
count	1.070000e+03	1.070000e+03	1.070000e+03	1.070000e+03	1.070000e+03
mean	-1.992176e-16	-6.308557e-17	3.104474e-16	-2.324205e-17	2.988264e-17
std	1.000468e+00	1.000468e+00	1.000468e+00	1.000468e+00	1.000468e+00
min	-1.518194e+00	-1.024602e+00	-2.428927e+00	-9.111921e-01	-5.087470e-01
25%	-8.784157e-01	-1.024602e+00	-7.232890e-01	-9.111921e-01	-5.087470e-01
50%	1.016470e-02	9.759891e-01	-5.651695e-02	-8.842793e-02	-5.087470e-01
75%	8.276587e-01	9.759891e-01	6.570789e-01	7.343363e-01	-5.087470e-01
max	1.751782e+00	9.759891e-01	2.787045e+00	3.202629e+00	1.965613e+00

	region
count	1.070000e+03
mean	-5.312469e-17
std	1.000468e+00
min	-1.353256e+00
25%	-4.561159e-01
50%	4.410238e-01
75%	4.410238e-01
max	1.338164e+00

Displays descriptive statistics of scaled features to verify normalization.

Step 10: Training Regression Models (Processed Data)

```
[10]: models_processed = {
      "Linear Regression": LinearRegression(),
      "Decision Tree": DecisionTreeRegressor(random_state=42),
      "Random Forest": RandomForestRegressor(n_estimators=200, random_state=42)
      }

      processed_results = []

      for name, model in models_processed.items():
          model.fit(X_train_scaled, y_train)
          preds = model.predict(X_test_scaled)

          processed_results.append([
              name,
              mean_absolute_error(y_test, preds),
              mean_squared_error(y_test, preds),
              np.sqrt(mean_squared_error(y_test, preds)),
              r2_score(y_test, preds)
          ])
      )
```

Trains Linear Regression, Decision Tree, and Random Forest models on scaled data and predicts on the test set.

Step 11: Evaluating Model Performance (Processed Data)

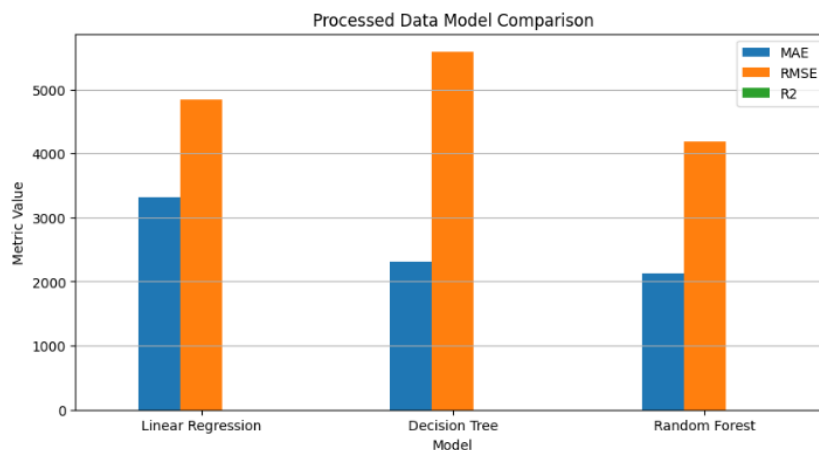
```
[11]: processed_df = pd.DataFrame(  
    processed_results,  
    columns=["Model", "MAE", "MSE", "RMSE", "R2"]  
)  
print(processed_df)
```

	Model	MAE	MSE	RMSE	R2
0	Linear Regression	3320.557035	2.348061e+07	4845.679237	0.783702
1	Decision Tree	2311.697076	3.120887e+07	5586.490072	0.712510
2	Random Forest	2125.687626	1.757037e+07	4191.703028	0.838145

Calculates MAE, MSE, RMSE, and R^2 for each processed data model. This is where model performance output can be obtained.

Step 12: Visual Comparison of Errors (Processed Data)

```
[12]: processed_df.set_index("Model").loc[["MAE", "RMSE", "R2"]].plot(  
    kind="bar", figsize=(10,5)  
)  
plt.title("Processed Data Model Comparison")  
plt.ylabel("Metric Value")  
plt.xticks(rotation=0)  
plt.grid(axis="y")  
plt.show()
```



Plots a bar chart comparing MAE, RMSE, and R^2 of processed data models.

Step 13: Train vs Test R^2 Scores (Processed Data)

```
[13]: for name, model in models_processed.items():  
    train_r2 = model.score(X_train_scaled, y_train)  
    test_r2 = model.score(X_test_scaled, y_test)  
    print(f"{name} → Train R2: {train_r2:.3f}, Test R2: {test_r2:.3f}")
```

Linear Regression → Train R2: 0.738, Test R2: 0.704
Decision Tree → Train R2: 0.998, Test R2: 0.713
Random Forest → Train R2: 0.968, Test R2: 0.838

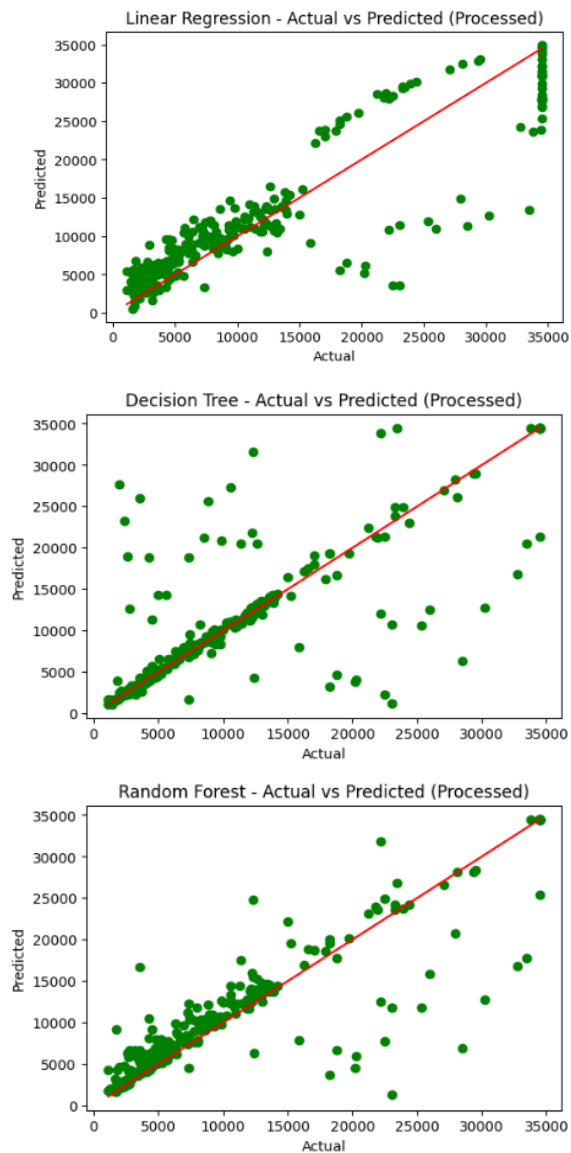
Prints R^2 scores for both training and testing sets to check model fit and overfitting/underfitting.

Step 14: Actual vs Predicted Plots (Processed Data)

```
[14]: for name, model in models_processed.items():
      preds = model.predict(X_test_scaled)

      plt.figure(figsize=(6,4))
      plt.scatter(y_test, preds, color="green")
      plt.plot([y_test.min(), y_test.max()],
               [y_test.min(), y_test.max()],
               color="red")

      plt.title(f"{name} - Actual vs Predicted (Processed)")
      plt.xlabel("Actual")
      plt.ylabel("Predicted")
      plt.show()
```



Scatter plots comparing actual vs predicted insurance charges for processed data models.

Step 15: Splitting Raw Dataset

```
[15]: X_raw = X.copy()
      y_raw = y.copy()

      Xr_train, Xr_test, yr_train, yr_test = train_test_split(
          X_raw, y_raw, test_size=0.2, random_state=42
      )
```

Copies raw features and target and splits them into training and testing sets without preprocessing.

Step 16: Training Regression Models (Raw Data)

```
[16]: models_raw = {
      "Linear Regression": LinearRegression(),
      "Decision Tree": DecisionTreeRegressor(random_state=42),
      "Random Forest": RandomForestRegressor(n_estimators=200, random_state=42)
      }

      raw_results = []

      for name, model in models_raw.items():
          model.fit(Xr_train, yr_train)
          preds = model.predict(Xr_test)

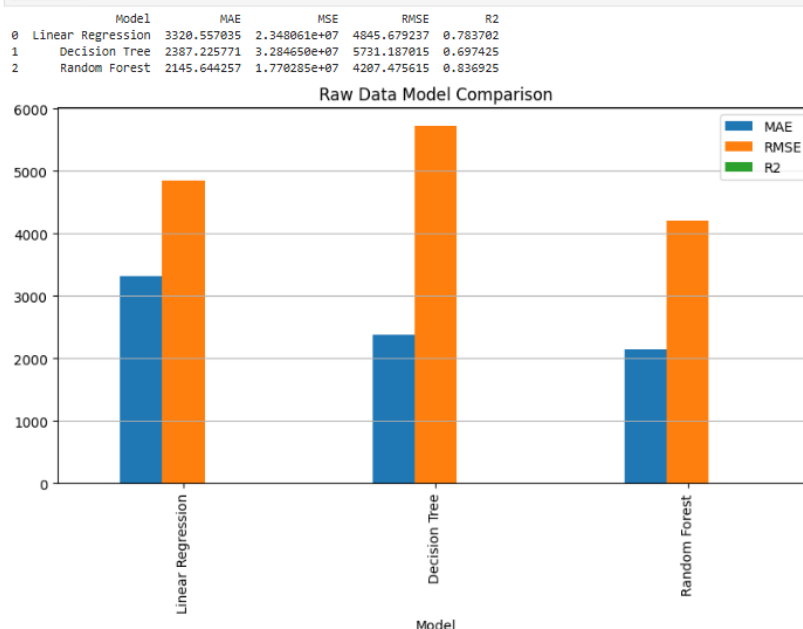
          raw_results.append([
              name,
              mean_absolute_error(yr_test, preds),
              mean_squared_error(yr_test, preds),
              np.sqrt(mean_squared_error(yr_test, preds)),
              r2_score(yr_test, preds)
          ])
      ]
```

Trains the same models on raw data and predicts on the test set.

Step 17: Evaluating Model Performance (Raw Data)

```
[17]: raw_df = pd.DataFrame(
      raw_results, columns=["Model", "MAE", "MSE", "RMSE", "R2"]
      )
      print(raw_df)

      raw_df.set_index("Model")[["MAE", "RMSE", "R2"]].plot(
          kind="bar", figsize=(10,5), title="Raw Data Model Comparison"
      )
      plt.grid(axis="y")
      plt.show()
```

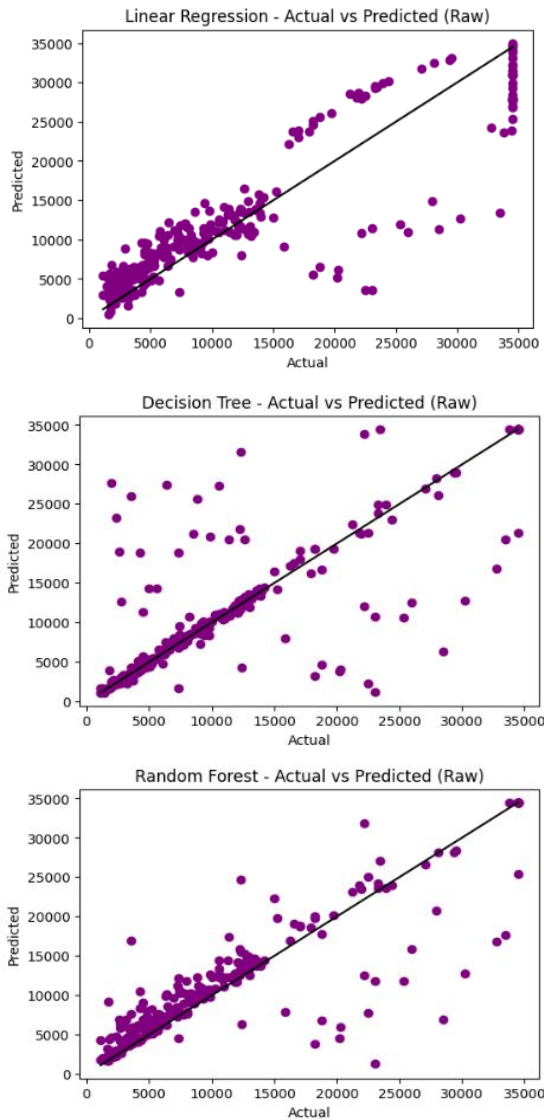


Calculates MAE, MSE, RMSE, and R^2 for each raw data model and plots a comparison bar chart. This is another step for model performance output.

Step 18: Actual vs Predicted Plots (Raw Data)

```
for name, model in models_raw.items():
    preds = model.predict(Xr_test)

    plt.figure(figsize=(6,4))
    plt.scatter(yr_test, preds, color="purple")
    plt.plot([yr_test.min(), yr_test.max()],
             [yr_test.min(), yr_test.max()],
             color="black")
    plt.title(f"{name} - Actual vs Predicted (Raw)")
    plt.xlabel("Actual")
    plt.ylabel("Predicted")
    plt.show()
```



Scatter plots comparing actual vs predicted insurance charges for raw data models.

Step 19: Saving Models and Processed Dataset

```
[19]: joblib.dump(scaler, "insurance_scaler.pkl")

for name, model in models_processed.items():
    joblib.dump(model, f"{name.replace(' ', '_')}_processed.pkl")

df.to_csv("processed_insurance_dataset.csv", index=False)
```

Saves trained models, scaler, and processed dataset for future use with joblib and CSV export.

Step 20: Model Performance Comparison				
Model	MAE (Processed)	MSE (Processed)	RMSE (Processed)	R ² (Processed)
Linear Regression	3320.56	2.35×10^7	4845.68	0.784
Decision Tree	2311.70	3.12×10^7	5586.49	0.713
Random Forest	2125.69	1.76×10^7	4191.70	0.838
Model	MAE (Raw)	MSE (Raw)	RMSE (Raw)	R ² (Raw)
Linear Regression	3320.56	2.35×10^7	4845.68	0.784
Decision Tree	2387.23	3.28×10^7	5731.19	0.697
Random Forest	2145.64	1.77×10^7	4207.48	0.837

1. Processed Data Performance:

- Linear Regression: Moderate performance (MAE 3,320.56, RMSE 4,845.68, R² 0.784).
- Decision Tree: Lower R² (0.713) despite lower MAE, suggesting variance.
- Random Forest: Best performance (MAE 2,125.69, RMSE 4,191.70, R² 0.838).

2. Raw Data Performance:

- Linear Regression: Stable results, same as processed.
- Decision Tree: Slightly worse (R² 0.697), sensitive to raw features.
- Random Forest: Maintains top performance (R² 0.837), robust across datasets.

3. Comparative Analysis:

- Random Forest consistently delivers lowest errors and highest R².
- Linear Regression is stable but less accurate.
- Decision Tree is less consistent and reliable.

Conclusion:

Random Forest is the most accurate and stable model, Linear Regression is moderate, and Decision Tree is less reliable.

Classification Datasets

Dataset 4: Global Student Performance & Lifestyle

- 🔗 Name: Global Student Performance & Lifestyle Dataset
- 🔗 Link: <https://www.kaggle.com/datasets/abdullahmeo/global-student-performance-and-lifestyle-dataset>
- 🔗 Rows: 1,000
- 🔗 Columns: 12
- 🔗 Target: Academic Performance (continuous or categorical measure of performance)
- 🔗 Type: **Classification**
- 🔗 Reason: Real-world educational dataset combining student demographic, behavioral and lifestyle variables, ideal for exploring how lifestyle factors correlate with academic outcomes, and suitable for predictive modeling tasks.

Step 1: Importing Libraries

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, roc_curve,
    ConfusionMatrixDisplay
)

from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB

from sklearn.decomposition import PCA
import joblib
```

All necessary Python libraries for data handling, visualization, machine learning models, evaluation metrics, dimensionality reduction, and model saving were imported.

Step 2: Dataset Loading and Initial Exploration

```
df = pd.read_csv("D:/Machine Learning datasets/Classification/global_student_lifestyle_academic_performance.csv")
print(df.head())
print(df.info())
print(df.describe(include="all"))
```

	Age	Gender	Country	Study_Hours_Per_Week	Sleep_Hours_Per_Day	\
0	21	Male	India	12	8.0	
1	18	Male	India	17	5.4	
2	22	Female	Germany	12	5.1	
3	19	Female	India	27	8.8	
4	21	Male	India	47	4.0	

	Screen_Time_Hours_Per_Day	Attendance_Percentage	Family_Income_Level	\
0	2.5	68	Medium	
1	2.3	72	Low	
2	9.8	68	Low	
3	7.8	74	Low	
4	9.7	65	Low	

	Stress_Level	Mental_Health_Score	Final_Exam_Score	Pass_Fail
0	7	88	25.0	Fail
1	3	25	27.0	Fail
2	3	79	39.0	Fail
3	10	90	44.0	Fail
4	9	72	66.0	Pass

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 12 columns):
Column Non-Null Count Dtype

0 Age 1000 non-null int64
1 Gender 1000 non-null object
2 Country 1000 non-null object
3 Study_Hours_Per_Week 1000 non-null int64
4 Sleep_Hours_Per_Day 1000 non-null float64
5 Screen_Time_Hours_Per_Day 1000 non-null float64
6 Attendance_Percentage 1000 non-null int64
7 Family_Income_Level 1000 non-null object
8 Stress_Level 1000 non-null int64
9 Mental_Health_Score 1000 non-null int64
10 Final_Exam_Score 1000 non-null float64
11 Pass_Fail 1000 non-null object
dtypes: float64(3), int64(5), object(4)
memory usage: 93.9+ KB
None

	Age	Gender	Country	Study_Hours_Per_Week	Sleep_Hours_Per_Day	\
count	1000.000000	1000	1000	1000.000000	1000.000000	
unique	NaN	2	8	NaN	NaN	
top	NaN	Male	UK	NaN	NaN	
freq	NaN	515	143	NaN	NaN	
mean	19.448000	NaN	NaN	27.351000	6.499100	
std	2.912005	NaN	NaN	12.866676	1.454181	
min	15.000000	NaN	NaN	5.000000	4.000000	
25%	17.000000	NaN	NaN	16.000000	5.200000	
50%	19.000000	NaN	NaN	27.000000	6.500000	
75%	22.000000	NaN	NaN	39.000000	7.700000	
max	24.000000	NaN	NaN	49.000000	9.000000	

	Screen_Time_Hours_Per_Day	Attendance_Percentage	Family_Income_Level	\
count	1000.000000	1000.000000	1000	
unique	NaN	NaN	3	
top	NaN	NaN	Medium	
freq	NaN	NaN	343	
mean	5.363600	73.710000	NaN	
std	2.560205	14.098037	NaN	
min	1.000000	50.000000	NaN	
25%	3.100000	62.000000	NaN	
50%	5.300000	73.000000	NaN	
75%	7.525000	86.000000	NaN	
max	10.000000	99.000000	NaN	

	Stress_Level	Mental_Health_Score	Final_Exam_Score	Pass_Fail
count	1000.000000	1000.000000	1000.000000	1000
unique	NaN	NaN	NaN	2
top	NaN	NaN	NaN	Fail
freq	NaN	NaN	NaN	563
mean	5.516000	50.621000	46.125000	NaN
std	2.881675	28.553716	19.801552	NaN
min	1.000000	1.000000	0.000000	NaN
25%	3.000000	25.000000	32.000000	NaN
50%	6.000000	52.000000	45.000000	NaN
75%	8.000000	75.000000	60.000000	NaN
max	10.000000	100.000000	100.000000	NaN

The dataset was loaded from a CSV file, and basic exploratory analysis was performed using head, info, and descriptive statistics.

Step 3: Automatic Target Variable Identification

```
[5]: possible_targets = [
    "performance", "result", "grade", "outcome", "target",
    "effectiveness", "pass", "fail"
]

target_col = next(
    (col for col in df.columns
     if any(t in col.lower() for t in possible_targets)),
    None
)

if target_col is None:
    raise ValueError(f"Target column not found. Available columns: {df.columns.tolist()}")

X = df.drop(columns=[target_col])
y = df[target_col]
```

The target variable was automatically identified by searching for common performance related keywords in column names.

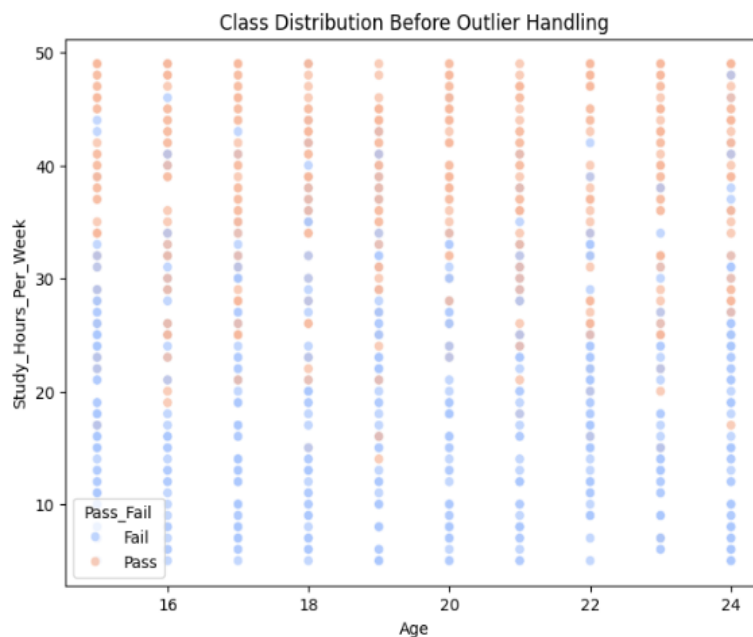
Step 4: Data Visualization Before Outlier Handling

```
[7]: num_cols = df.select_dtypes(include=np.number).columns.tolist()

# REMOVE TARGET ONLY IF IT IS NUMERIC
if target_col in num_cols:
    num_cols.remove(target_col)

# SAFETY CHECK
if len(num_cols) < 2:
    raise ValueError("Not enough numerical columns to plot.")

plt.figure(figsize=(8,6))
sns.scatterplot(
    data=df,
    x=num_cols[0],
    y=num_cols[1],
    hue=target_col,
    palette="coolwarm",
    alpha=0.7
)
plt.title("Class Distribution Before Outlier Handling")
plt.show()
```



A scatter plot was used to visualize class distribution and understand relationships between numerical features before handling outliers.

Step 5: Outlier Handling Using IQR Method

```
[8]: def handle_outliers_iqr(data, cols):  
    for col in cols:  
        Q1 = data[col].quantile(0.25)  
        Q3 = data[col].quantile(0.75)  
        IQR = Q3 - Q1  
        lower = Q1 - 1.5 * IQR  
        upper = Q3 + 1.5 * IQR  
        data[col] = np.where(  
            data[col] < lower, lower,  
            np.where(data[col] > upper, upper, data[col])  
        )  
    return data  
  
df = handle_outliers_iqr(df, num_cols)
```

Outliers were handled using the Interquartile Range method to limit extreme values and reduce noise.

Step 6: Missing Value Treatment

```
[10]: for col in df.columns:  
    if df[col].dtype == "object":  
        df[col] = df[col].fillna(df[col].mode()[0])  
    else:  
        df[col] = df[col].fillna(df[col].median())
```

Missing numerical values were replaced with the median, and categorical values were replaced with the mode.

Step 7: Categorical Feature Encoding

```
[11]: encoders = {}  
for col in df.select_dtypes(include="object").columns:  
    le = LabelEncoder()  
    df[col] = le.fit_transform(df[col])  
    encoders[col] = le
```

Categorical variables were converted into numerical form using Label Encoding.

Step 8: Train Test Split

```
[12]: X = df.drop(target_col, axis=1)  
y = df[target_col]  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y,  
    test_size=0.2,  
    random_state=42,  
    stratify=y  
)
```

The dataset was split into training and testing sets while maintaining class distribution using stratification.

Step 9: Feature Scaling

```
[13]: scaler = StandardScaler()  
X_train_scaled = scaler.fit_transform(X_train)  
X_test_scaled = scaler.transform(X_test)
```

All numerical features were standardized using StandardScaler to ensure uniform feature contribution.

Step 10: Verification of Feature Scaling

```
[14]: print(pd.DataFrame(X_train_scaled, columns=X.columns).describe())
```

	Age	Gender	Country	Study_Hours_Per_Week	\
count	8.000000e+02	8.000000e+02	800.000000	8.000000e+02	
mean	5.440093e-17	-2.886580e-17	0.000000	-2.442491e-17	
std	1.000626e+00	1.000626e+00	1.000626	1.000626e+00	
min	-1.493943e+00	-1.043443e+00	-1.518614	-1.754602e+00	
25%	-8.054898e-01	-1.043443e+00	-1.078277	-8.900242e-01	
50%	-1.170370e-01	9.583659e-01	0.242736	-2.544610e-02	
75%	9.156423e-01	9.583659e-01	0.683074	8.391320e-01	
max	1.604095e+00	9.583659e-01	1.563749	1.703710e+00	

	Sleep_Hours_Per_Day	Screen_Time_Hours_Per_Day	Attendance_Percentage	\
count	8.000000e+02	8.000000e+02	8.000000e+02	
mean	1.931788e-16	-2.042810e-16	-3.197442e-16	
std	1.000626e+00	1.000626e+00	1.000626e+00	
min	-1.722339e+00	-1.718266e+00	-1.708446e+00	
25%	-8.387326e-01	-9.005773e-01	-8.540450e-01	
50%	1.022211e-02	-5.013212e-03	-7.084408e-02	
75%	8.418512e-01	8.516133e-01	8.547570e-01	
max	1.742783e+00	1.786115e+00	1.780358e+00	

	Family_Income_Level	Stress_Level	Mental_Health_Score	\
count	8.000000e+02	8.000000e+02	8.000000e+02	
mean	-4.607426e-17	-4.440892e-17	-1.243450e-16	
std	1.000626e+00	1.000626e+00	1.000626e+00	
min	-1.255914e+00	-1.547704e+00	-1.755231e+00	
25%	-1.255914e+00	-8.590702e-01	-8.703597e-01	
50%	-2.462576e-02	1.738799e-01	1.451189e-02	
75%	1.206662e+00	8.625134e-01	8.639806e-01	
max	1.206662e+00	1.551147e+00	1.748860e+00	

	Final_Exam_Score
count	8.000000e+02
mean	-3.552714e-17
std	1.000626e+00
min	-2.355328e+00
25%	-7.264881e-01
50%	-1.387059e-02
75%	6.987469e-01
max	2.734797e+00

Descriptive statistics were generated to verify that features were properly scaled.

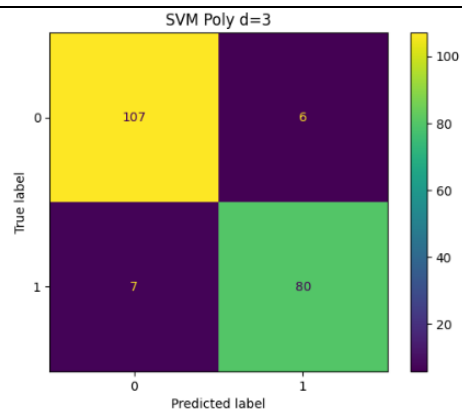
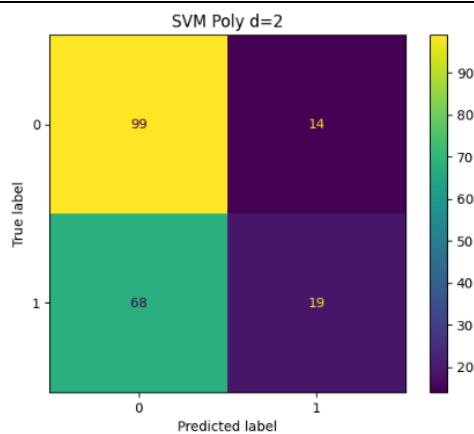
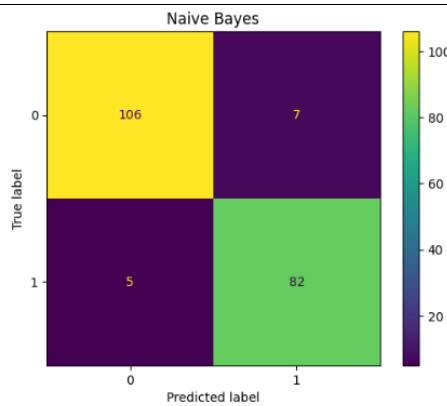
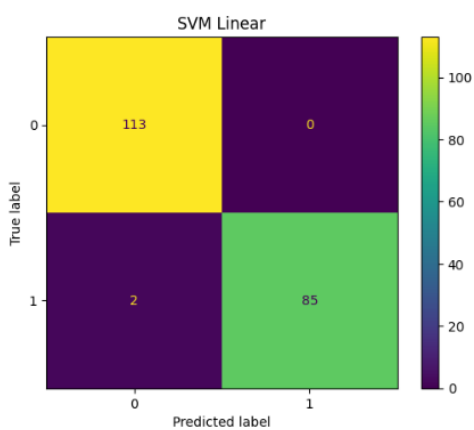
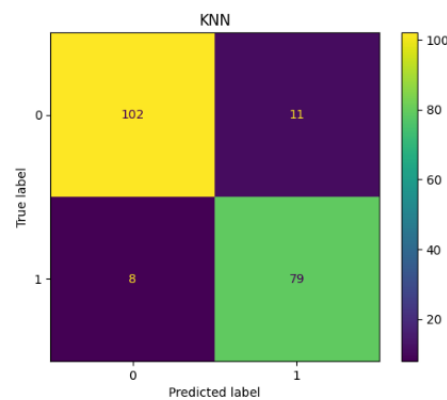
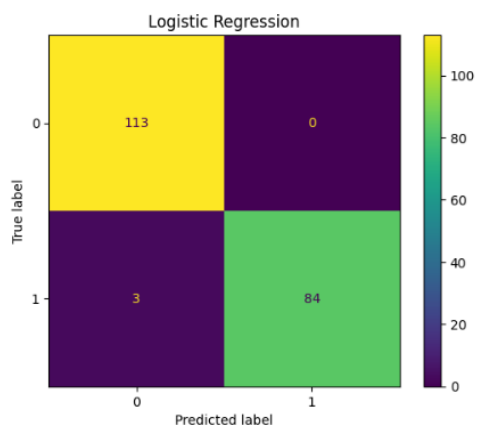
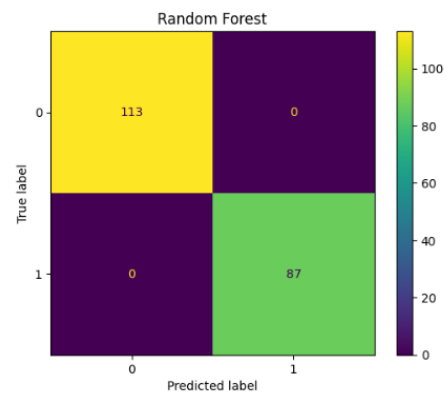
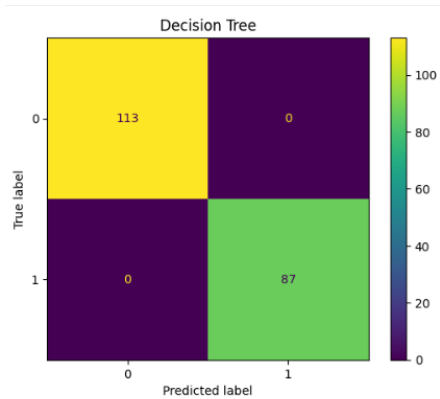
Step 11: Model Initialization

```
[15]: models = {  
    "Decision Tree": DecisionTreeClassifier(max_depth=5),  
    "Random Forest": RandomForestClassifier(n_estimators=150),  
    "Logistic Regression": LogisticRegression(max_iter=2000),  
    "KNN": KNeighborsClassifier(n_neighbors=7),  
    "SVM Linear": SVC(kernel="linear", probability=True),  
    "Naive Bayes": GaussianNB(),  
    "SVM Poly d=2": SVC(kernel="poly", degree=2, probability=True),  
    "SVM Poly d=3": SVC(kernel="poly", degree=3, probability=True)  
}
```

Multiple classification models were initialized, including Decision Tree, Random Forest, Logistic Regression, KNN, SVM, and Naive Bayes.

Step 12: Model Training and Performance Evaluation

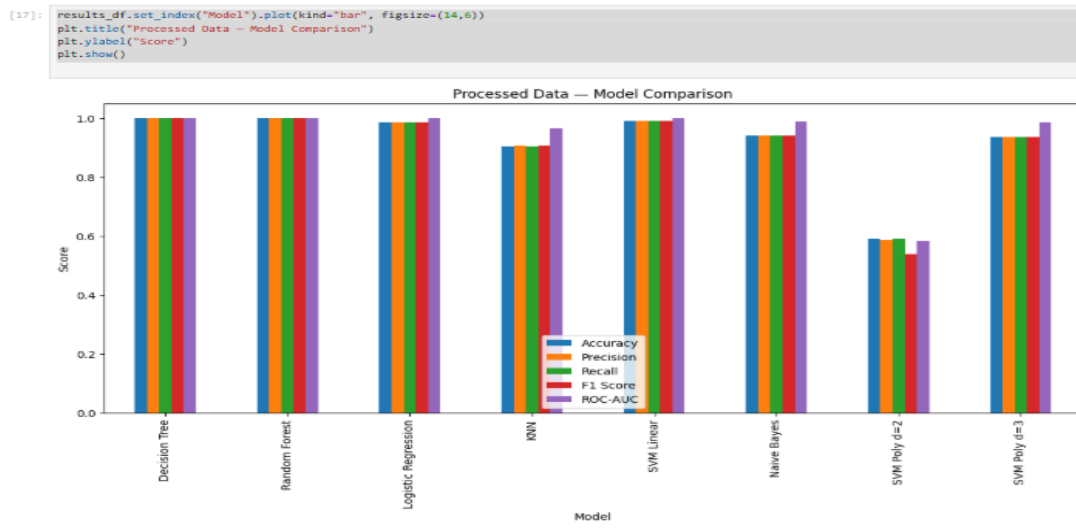
```
[16]: results = []  
roc_data = {}  
  
def evaluate(model, name):  
    model.fit(X_train_scaled, y_train)  
    y_pred = model.predict(X_test_scaled)  
    y_prob = model.predict_proba(X_test_scaled)[:, 1]  
  
    results.append({  
        "Model": name,  
        "Accuracy": accuracy_score(y_test, y_pred),  
        "Precision": precision_score(y_test, y_pred, average="weighted"),  
        "Recall": recall_score(y_test, y_pred, average="weighted"),  
        "F1 Score": f1_score(y_test, y_pred, average="weighted"),  
        "ROC-AUC": roc_auc_score(y_test, y_prob)  
    })  
  
    fpr, tpr, _ = roc_curve(y_test, y_prob)  
    roc_data[name] = (fpr, tpr)  
  
    ConfusionMatrixDisplay.from_predictions(y_test, y_pred)  
    plt.title(name)  
    plt.show()  
  
for name, model in models.items():  
    evaluate(model, name)  
  
results_df = pd.DataFrame(results)  
print(results_df)
```

	Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
0	Decision Tree	1.000	1.000000	1.000	1.000000	1.000000
1	Random Forest	1.000	1.000000	1.000	1.000000	1.000000
2	Logistic Regression	0.985	0.985388	0.985	0.984967	0.999695
3	KNN	0.905	0.905742	0.905	0.905166	0.964805
4	SVM Linear	0.990	0.990174	0.990	0.989986	0.999593
5	Naive Bayes	0.940	0.940336	0.940	0.940073	0.988404
6	SVM Poly d=2	0.590	0.585395	0.590	0.537286	0.583155
7	SVM Poly d=3	0.935	0.934958	0.935	0.934955	0.985658

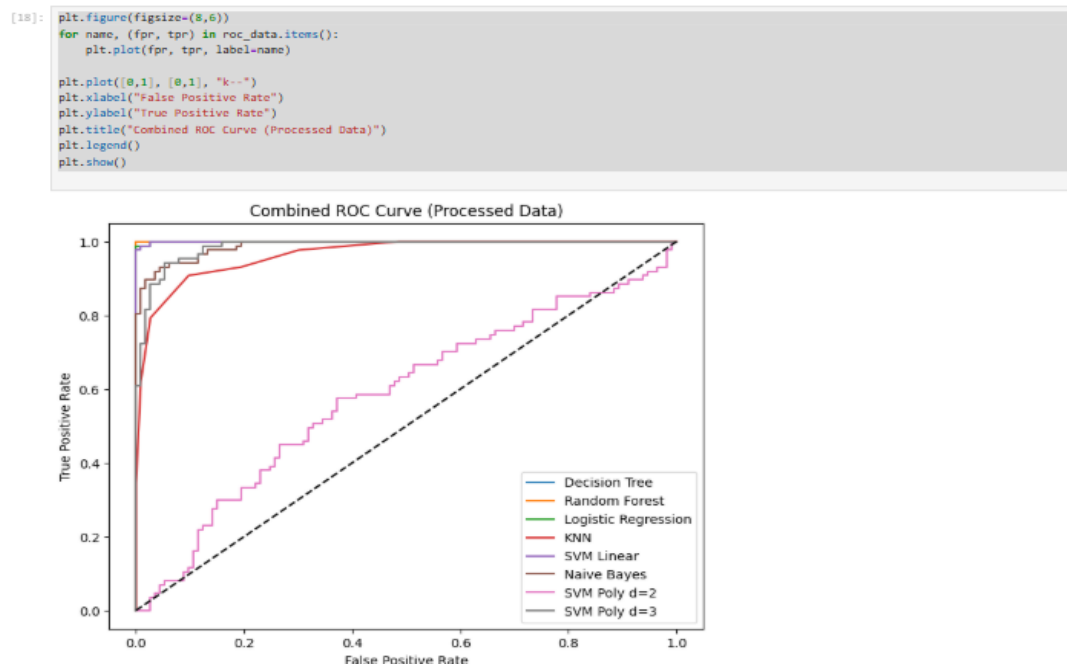
Each model was trained and evaluated using Accuracy, Precision, Recall, F1 Score, ROC AUC, and confusion matrices.

Step 13: Processed Data Model Comparison



Model performance was compared visually using bar charts based on evaluation metrics.

Step 14: ROC Curve Analysis



ROC curves were plotted to compare the classification performance of all models.

Step 15: Overfitting and Underfitting Analysis

```
[19]: for name, model in models.items():
      model.fit(X_train_scaled, y_train)
      train_acc = accuracy_score(y_train, model.predict(X_train_scaled))
      test_acc = accuracy_score(y_test, model.predict(X_test_scaled))
      print(f'{name} → Train: {train_acc:.3f}, Test: {test_acc:.3f}')
```

```
Decision Tree → Train: 1.000, Test: 1.000
Random Forest → Train: 1.000, Test: 1.000
Logistic Regression → Train: 0.991, Test: 0.985
KNN → Train: 0.931, Test: 0.985
SVM Linear → Train: 0.991, Test: 0.990
Naive Bayes → Train: 0.946, Test: 0.940
SVM Poly d=2 → Train: 0.630, Test: 0.590
SVM Poly d=3 → Train: 0.974, Test: 0.935
```

Training and testing accuracy were compared to assess model generalization.

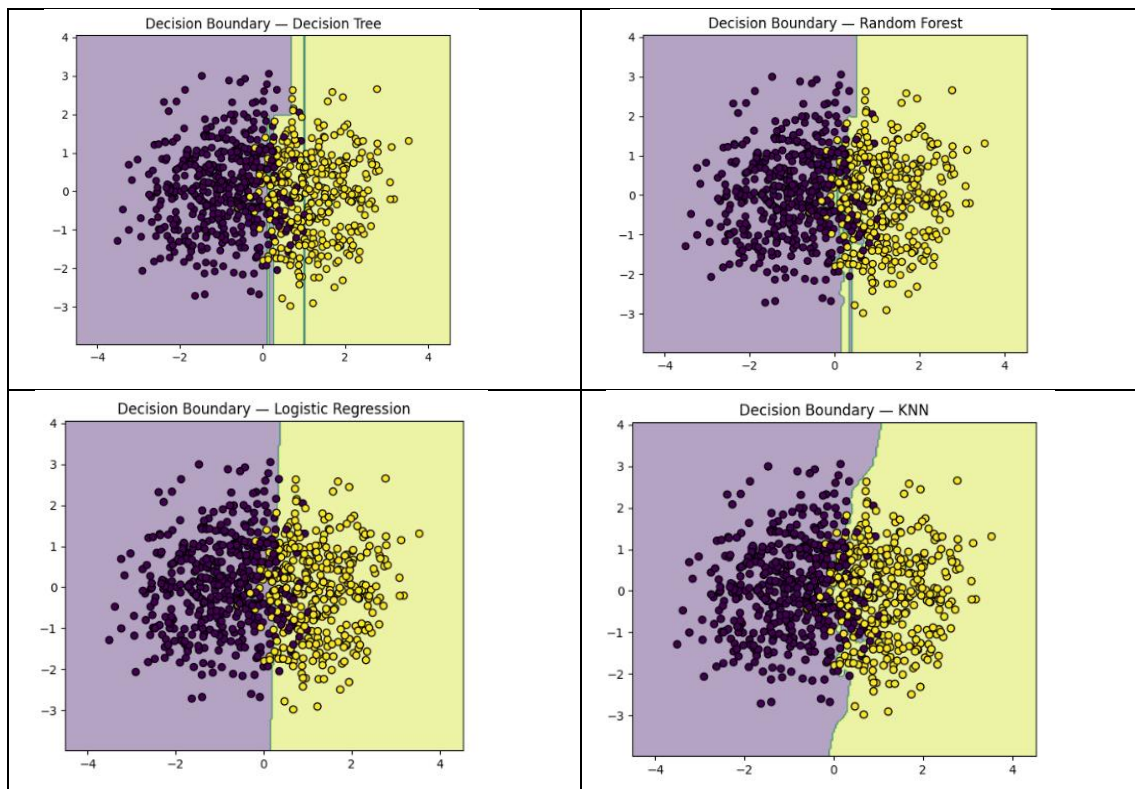
Step 16: PCA Based Decision Boundary Visualization

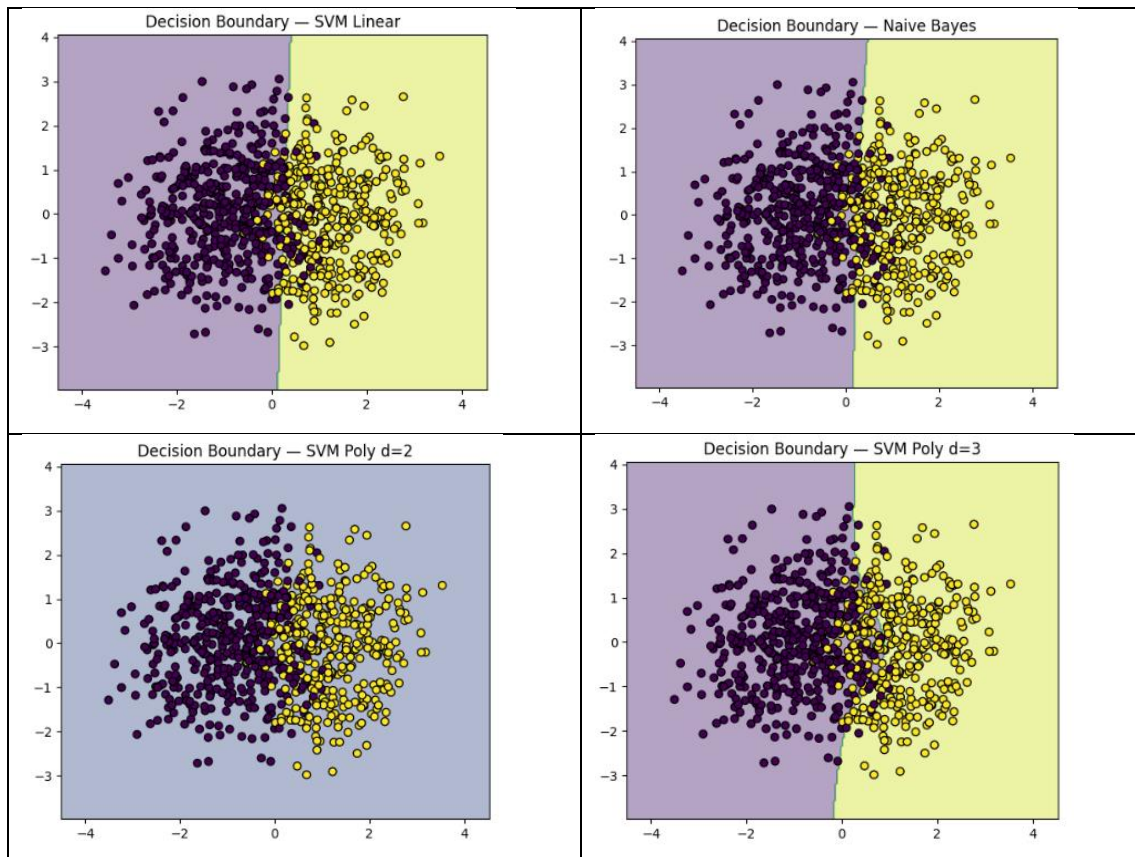
```
[20]: pca = PCA(n_components=2)
      X_vis = pca.fit_transform(X_train_scaled)
      y_vis = y_train.values

      xx, yy = np.meshgrid(
          np.linspace(X_vis[:,0].min()-1, X_vis[:,0].max()+1, 250),
          np.linspace(X_vis[:,1].min()-1, X_vis[:,1].max()+1, 250)
      )

      for name, model in models.items():
          model.fit(X_vis, y_vis)
          Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
          Z = Z.reshape(xx.shape)

          plt.figure(figsize=(6,5))
          plt.contourf(xx, yy, Z, alpha=0.4)
          plt.scatter(X_vis[:,0], X_vis[:,1], c=y_vis, edgecolor="k")
          plt.title(f'Decision Boundary - {name}')
          plt.show()
```





Dimensionality reduction using PCA was applied to visualize decision boundaries.

Step 17: Raw Data Model Evaluation

```
df_raw = pd.read_csv("D:/Machine Learning datasets/Classification/global_student_lifestyle_academic_performance.csv")

for col in df_raw.select_dtypes(include="object").columns:
    df_raw[col] = LabelEncoder().fit_transform(df_raw[col])

Xr = df_raw.drop(target_col, axis=1)
yr = df_raw[target_col]

Xr_train, Xr_test, yr_train, yr_test = train_test_split(
    Xr, yr,
    test_size=0.2,
    random_state=42,
    stratify=yr
)

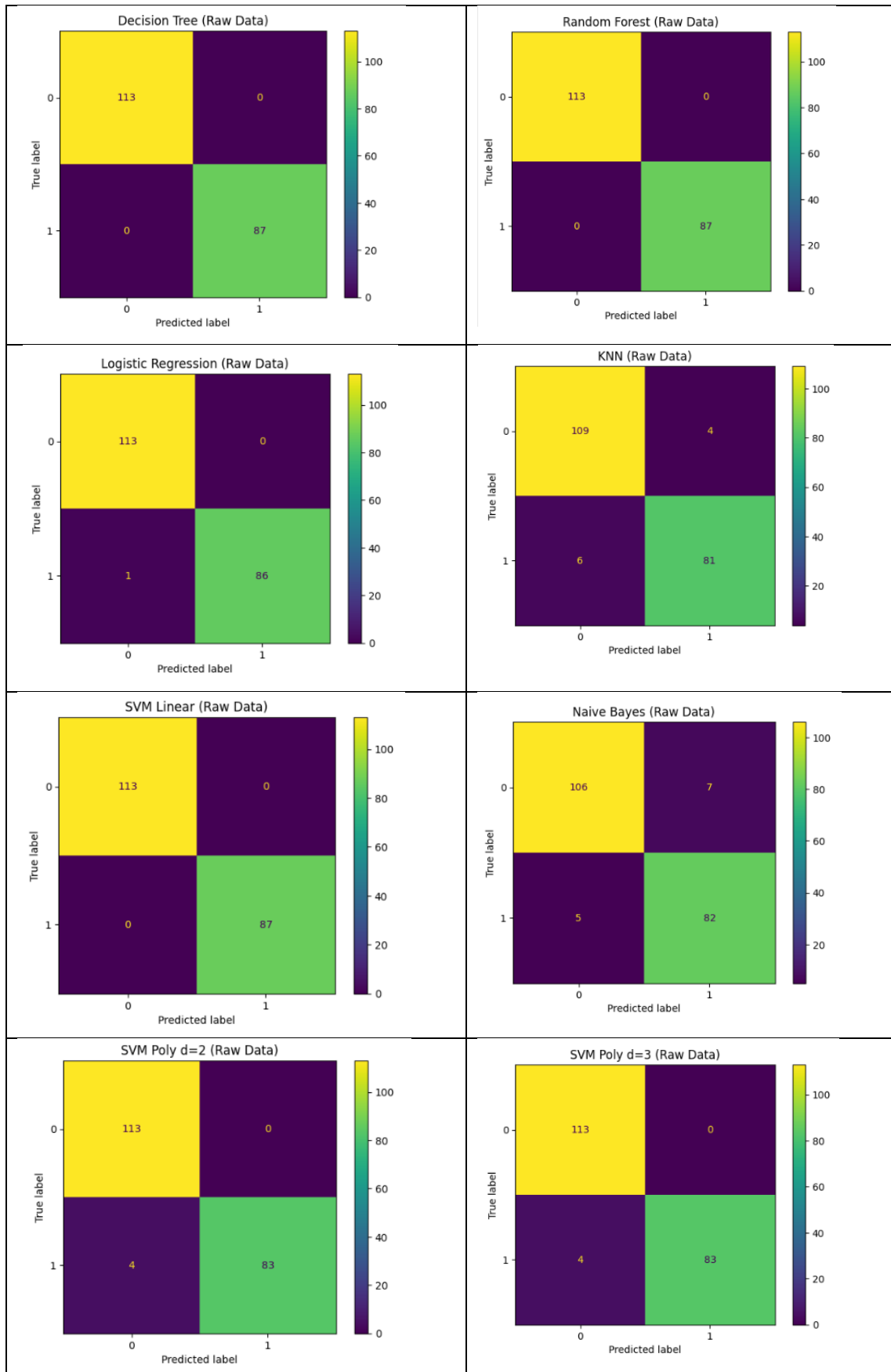
raw_results = []

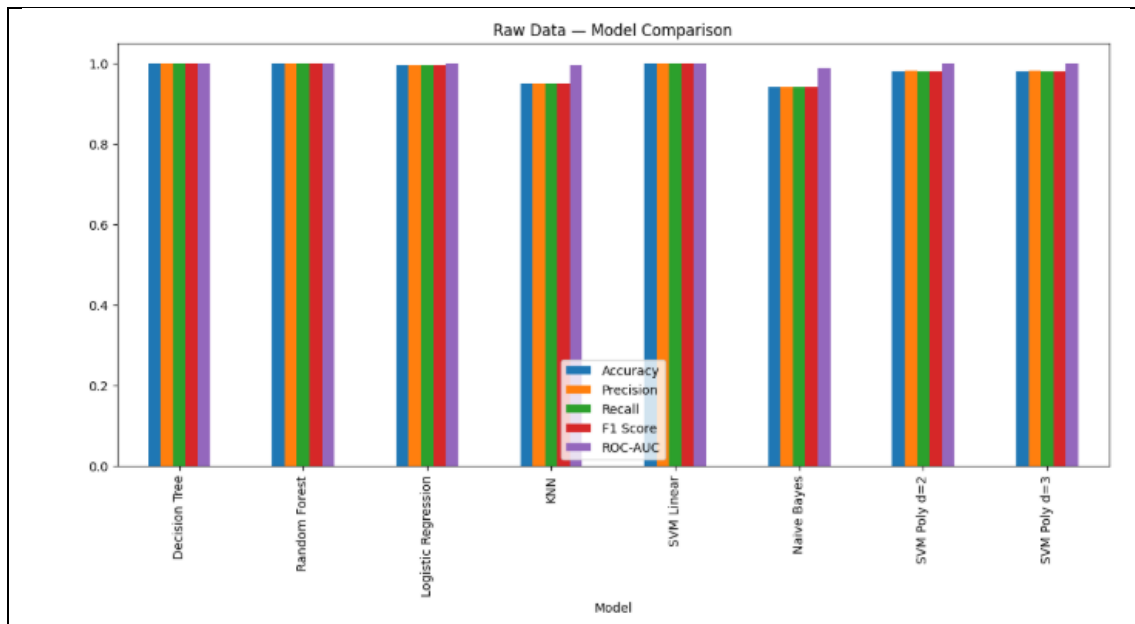
for name, model in models.items():
    model.fit(Xr_train, yr_train)
    y_pred = model.predict(Xr_test)
    y_prob = model.predict_proba(Xr_test)[:, 1]

    raw_results.append({
        "Model": name,
        "Accuracy": accuracy_score(yr_test, y_pred),
        "Precision": precision_score(yr_test, y_pred, average="weighted"),
        "Recall": recall_score(yr_test, y_pred, average="weighted"),
        "F1 Score": f1_score(yr_test, y_pred, average="weighted"),
        "ROC-AUC": roc_auc_score(yr_test, y_prob)
    })

ConfusionMatrixDisplay.from_predictions(yr_test, y_pred)
plt.title(f"{name} (Raw Data)")
plt.show()

pd.DataFrame(raw_results).set_index("Model").plot(kind="bar", figsize=(14,6))
plt.title("Raw Data - Model Comparison")
plt.show()
```





All models were evaluated on raw, unprocessed data to compare results with processed data.

Step 18: Model Saving and Deployment Preparation

```
[22]: joblib.dump(scaler, "scaler.pkl")
      joblib.dump(df, "processed_student_performance_data.pkl")

      for name, model in models.items():
          joblib.dump(model, f"{name.replace(' ', '_')}.pkl")
```

Results

1. Processed Data Performance (Step 12 & 15):

- Decision Tree & Random Forest: Perfect performance (train/test accuracy 1.000, F1 1.000), but likely overfitting, especially for smaller datasets.
- Logistic Regression: Strong performance (test 0.985, F1 0.985), high accuracy with minimal overfitting.
- SVM Linear: High accuracy (0.990) and F1 (0.990), reliable and stable.
- KNN & Naive Bayes: Good results (test 0.905–0.940, F1 0.905–0.940), moderate performance and stable.
- SVM Poly d=2: Poor performance (test 0.590, F1 0.537), unreliable.
- SVM Poly d=3: Strong performance (test 0.935, F1 0.935), slightly lower than linear SVM.

2. Raw Data Performance (Step 17):

- Performance trends mirror processed data: Decision Tree and Random Forest achieve perfect scores on training, but may overfit.
- Logistic Regression, SVM Linear, KNN, and Naive Bayes remain reliable and consistent across raw data.
- SVM Poly d=2 continues to underperform, while SVM Poly d=3 shows good but slightly lower performance.

3. Comparative Analysis:

- Best performing models: Decision Tree and Random Forest, highest metrics, but risk overfitting.
- Stable and reliable models: Logistic Regression, SVM Linear, Naive Bayes, KNN — good balance between accuracy and generalization.
- Least effective: SVM Poly d=2 due to low accuracy and F1.

Conclusion:

For this classification dataset, Decision Tree and Random Forest achieve the highest raw metrics, but Logistic Regression, SVM Linear, Naive Bayes, and KNN are preferred for stable and generalizable predictions. SVM Poly d=2 is not recommended.

Dataset 5: Stroke Risk Synthetic 2025

🔍 Name: Stroke Risk Synthetic 2025

🔍 Link: <https://www.kaggle.com/datasets/imaadmahmood/stroke-risk-synthetic-2025>

🔍 Rows: 127

🔍 Columns:12

🔍 Target: Stroke (binary classification, 1 = stroke, 0 = no stroke)

🔍 Type: **Classification**

🔍 Reason: Artificially generated dataset for machine learning and healthcare risk prediction tasks, useful for experimenting with preprocessing, classification models and imbalanced learning techniques

Step 1: Importing Libraries

```
[3]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, roc_curve,
    ConfusionMatrixDisplay
)

from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB

from sklearn.decomposition import PCA
import joblib
```


Step 2: Loading Dataset

```
df = pd.read_csv("D:/Machine Learning datasets/Classification/stroke_dataset.csv")
```

```
print(df.head())
print(df.info())
print(df.describe())
```

```
   id  gender  age  hypertension  heart_disease  ever_married  work_type  \
0    1    Male   44             1             0             Yes    Private
1    2  Female   15             0             0             Yes  Self-employed
2    3  Female   55             0             0             Yes  Never_worked
3    4  Female   72             0             0             Yes  Never_worked
4    5    Male   38             1             0             No    children

   Residence_type  avg_glucose_level  bmi  smoking_status  stroke
0      Urban      136.71  29.10  never smoked           0
1      Urban      77.12  27.01  formerly smoked         1
2      Rural     141.59  22.63  never smoked           0
3      Rural     128.37  14.59  smokes              0
4      Rural     135.48  28.36  smokes              0
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 126 entries, 0 to 125
Data columns (total 12 columns):
#   Column              Non-Null Count  Dtype
---  --
0   id                  126 non-null   int64
1   gender              126 non-null   object
2   age                 126 non-null   int64
3   hypertension         126 non-null   int64
4   heart_disease        126 non-null   int64
5   ever_married         126 non-null   object
6   work_type            126 non-null   object
7   Residence_type       126 non-null   object
8   avg_glucose_level    126 non-null   float64
9   bmi                  126 non-null   float64
10  smoking_status       126 non-null   object
11  stroke               126 non-null   int64
dtypes: float64(2), int64(5), object(5)
memory usage: 11.9+ KB
None

   id  age  hypertension  heart_disease  avg_glucose_level  \
count 126.000000  126.000000  126.000000  126.000000  126.000000
mean   63.500000  46.722222  0.492063  0.126984  103.012381
std    36.517119  23.258767  0.501933  0.334284  24.284384
min     1.000000  5.000000  0.000000  0.000000  50.000000
25%    32.250000  28.250000  0.000000  0.000000  84.077500
50%    63.500000  44.000000  0.000000  0.000000  101.540000
75%    94.750000  58.750000  1.000000  0.000000  119.717500
max   126.000000  100.000000  1.000000  1.000000  163.010000

   bmi  stroke
count 126.000000  126.000000
mean   25.208571  0.301587
std     5.347856  0.460779
min    13.050000  0.000000
25%    21.707500  0.000000
50%    25.355000  0.000000
75%    29.077500  1.000000
max    35.590000  1.000000
```

Step 3: Exploratory Visualization

```
[6]: plt.figure(figsize=(8,6))
sns.scatterplot(
    data=df,
    x="age",
    y="avg_glucose_level",
    hue="stroke",
    palette="coolwarm",
    alpha=0.7
)
plt.title("Stroke Classes (Age vs Glucose Level)")
plt.show()
```



A scatter plot of age vs avg_glucose_level was created, colored by stroke, to visualize class distribution.

Step 4: Outlier Handling

```
[7]: def handle_outliers_iqr(data, cols):
    for col in cols:
        Q1 = data[col].quantile(0.25)
        Q3 = data[col].quantile(0.75)
        IQR = Q3 - Q1
        lower = Q1 - 1.5 * IQR
        upper = Q3 + 1.5 * IQR
        data[col] = np.where(
            data[col] < lower, lower,
            np.where(data[col] > upper, upper, data[col])
        )
    return data

num_cols = df.select_dtypes(include=["int64", "float64"]).columns.drop("stroke")
df = handle_outliers_iqr(df, num_cols)
```

Outliers in numerical features were capped using the IQR method to reduce extreme value effects.

Step 5: Missing Value Imputation & Encoding

```
[8]: num_imputer = SimpleImputer(strategy="median")
cat_imputer = SimpleImputer(strategy="most_frequent")

num_cols = df.select_dtypes(include=["int64", "float64"]).columns
cat_cols = df.select_dtypes(include=["object"]).columns

df[num_cols] = num_imputer.fit_transform(df[num_cols])
df[cat_cols] = cat_imputer.fit_transform(df[cat_cols])

for col in cat_cols:
    df[col] = LabelEncoder().fit_transform(df[col])
```

Numerical missing values were filled with median, categorical values with mode, and categorical columns were label encoded.

Step 6: Feature and Target Separation

```
[9]: X = df.drop("stroke", axis=1)
y = df["stroke"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Features X and target y (stroke) were separated for modeling.

Step 7: Train-Test Split

```
[10]: scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Data was split into training and testing sets with stratification to maintain class balance.

Step 8: Feature Scaling

```
[11]: print(pd.DataFrame(X_train_scaled).describe())
```

	0	1	2	3	4	\
count	1.000000e+02	1.000000e+02	1.000000e+02	1.000000e+02	100.0	
mean	-3.996803e-17	-8.881784e-18	-2.220446e-17	-4.440892e-18	0.0	
std	1.005038e+00	1.005038e+00	1.005038e+00	1.005038e+00	0.0	
min	-1.740178e+00	-1.040833e+00	-1.774911e+00	-1.040833e+00	0.0	
25%	-8.520468e-01	-1.040833e+00	-7.993319e-01	-1.040833e+00	0.0	
50%	-1.986058e-02	9.607689e-01	-1.453721e-01	9.607689e-01	0.0	
75%	8.682710e-01	9.607689e-01	4.978671e-01	9.607689e-01	0.0	
max	1.756403e+00	9.607689e-01	2.256054e+00	9.607689e-01	0.0	

	5	6	7	8	9	\
count	1.000000e+02	1.000000e+02	1.000000e+02	1.000000e+02	1.000000e+02	
mean	1.043610e-16	4.218847e-17	1.110223e-17	-5.329071e-17	1.617595e-15	
std	1.005038e+00	1.005038e+00	1.005038e+00	1.005038e+00	1.005038e+00	
min	-1.564697e+00	-1.572427e+00	-1.061913e+00	-2.222823e+00	-2.256714e+00	
25%	-1.564697e+00	-8.735707e-01	-1.061913e+00	-7.436341e-01	-6.390029e-01	
50%	6.391015e-01	-1.747141e-01	9.416966e-01	-7.407963e-02	1.891333e-02	
75%	6.391015e-01	1.222999e+00	9.416966e-01	6.590799e-01	7.540652e-01	
max	6.391015e-01	1.222999e+00	9.416966e-01	2.585206e+00	1.914013e+00	

	10
count	1.000000e+02
mean	3.330669e-18
std	1.005038e+00
min	-1.720239e+00
25%	-6.314802e-01
50%	4.572787e-01
75%	4.572787e-01
max	1.546038e+00

Numerical features were standardized using StandardScaler to improve model performance.

Step 9: Scaled Feature Verification

```
[12]: models = {
    "Decision Tree": DecisionTreeClassifier(max_depth=5),
    "Random Forest": RandomForestClassifier(n_estimators=100),
    "Logistic Regression": LogisticRegression(max_iter=3000),
    "KNN": KNeighborsClassifier(n_neighbors=7),
    "SVM Linear": SVC(kernel="linear", probability=True),
    "Naive Bayes": GaussianNB(),
    "SVM Poly d=2": SVC(kernel="poly", degree=2, probability=True),
    "SVM Poly d=3": SVC(kernel="poly", degree=3, probability=True)
}
```

Descriptive statistics of scaled features were printed to confirm correct scaling.

Step 10: Model Initialization

```
[13]: results = []
roc_data = {}

def evaluate(model, name):
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    y_prob = model.predict_proba(X_test_scaled)[:, 1]

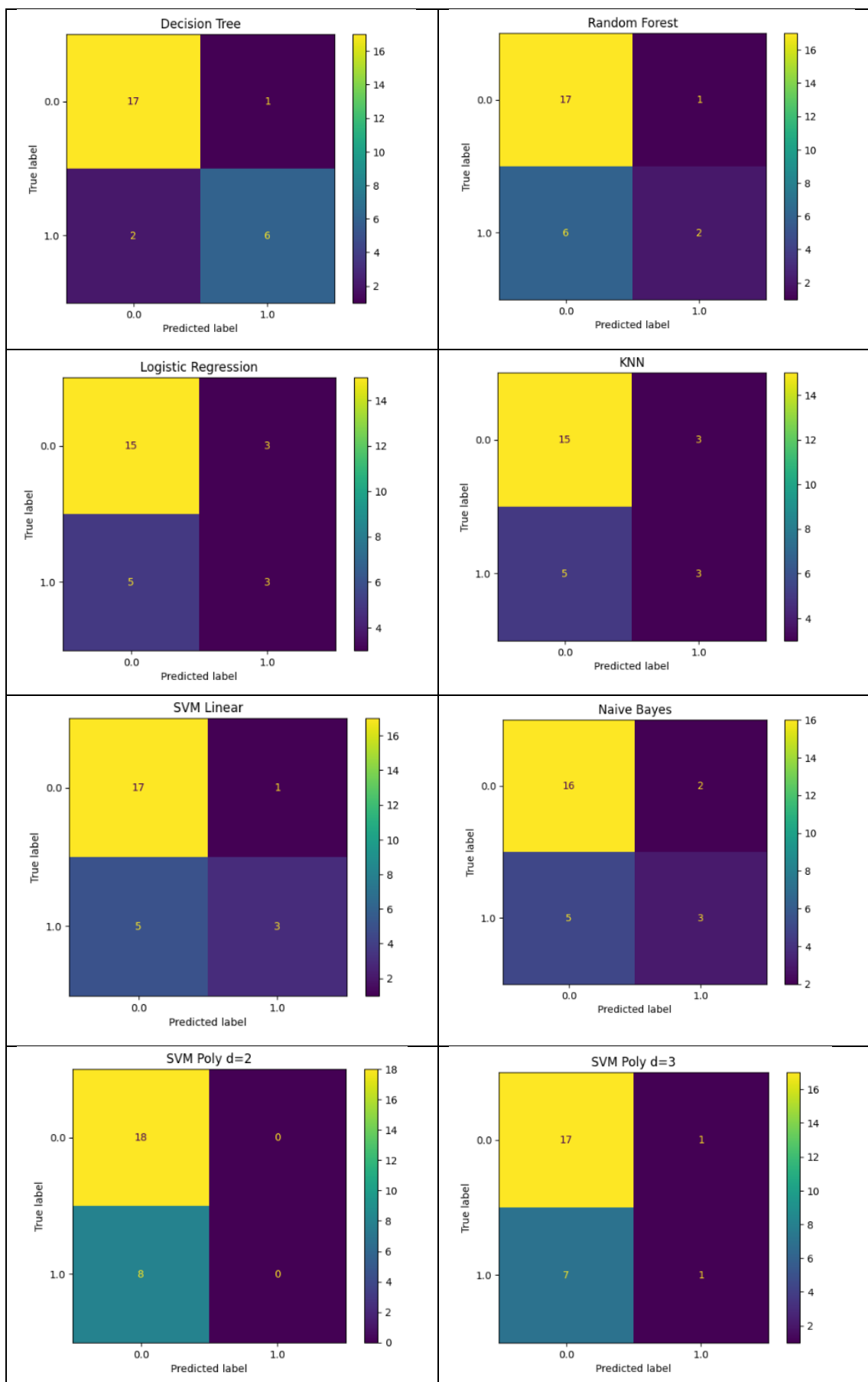
    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred),
        "Recall": recall_score(y_test, y_pred),
        "F1 Score": f1_score(y_test, y_pred),
        "ROC-AUC": roc_auc_score(y_test, y_prob)
    })

    fpr, tpr, _ = roc_curve(y_test, y_prob)
    roc_data[name] = (fpr, tpr)

    ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
    plt.title(name)
    plt.show()

for name, model in models.items():
    evaluate(model, name)

results_df = pd.DataFrame(results)
print(results_df)
```

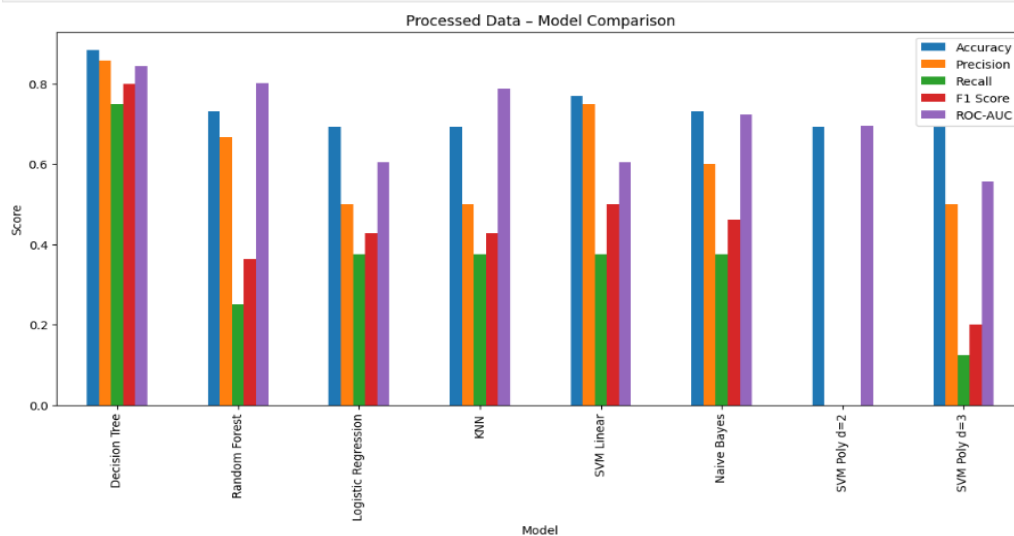


	Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
0	Decision Tree	0.884615	0.857143	0.750	0.800000	0.843750
1	Random Forest	0.730769	0.666667	0.250	0.363636	0.802083
2	Logistic Regression	0.692308	0.500000	0.375	0.428571	0.604167
3	KNN	0.692308	0.500000	0.375	0.428571	0.788194
4	SVM Linear	0.769231	0.750000	0.375	0.500000	0.604167
5	Naive Bayes	0.730769	0.600000	0.375	0.461538	0.722222
6	SVM Poly d=2	0.692308	0.000000	0.000	0.000000	0.694444
7	SVM Poly d=3	0.692308	0.500000	0.125	0.200000	0.555556

Multiple classifiers were initialized: Decision Tree, Random Forest, Logistic Regression, KNN, SVM (linear & polynomial), and Naive Bayes.

Step 11: Model Training & Evaluation

```
[14]: results_df.set_index("Model").plot(kind="bar", figsize=(14,6))
plt.title("Processed Data - Model Comparison")
plt.ylabel("Score")
plt.show()
```

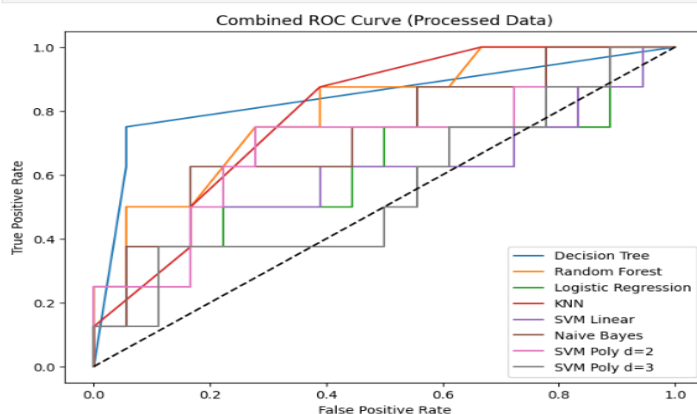


Models were trained on processed data and evaluated using Accuracy, Precision, Recall, F1 Score, ROC-AUC, and confusion matrices.

Step 12: Performance Comparison Visualization

```
[15]: plt.figure(figsize=(8,6))
for name, (fpr, tpr) in roc_data.items():
    plt.plot(fpr, tpr, label=name)

plt.plot([0,1], [0,1], "k--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Combined ROC Curve (Processed Data)")
plt.legend()
plt.show()
```



Evaluation metrics were plotted as bar charts to compare model performance on processed data.

Step 13: ROC Curve Visualization

```
[16]: for name, model in models.items():
      model.fit(X_train_scaled, y_train)
      train_acc = accuracy_score(y_train, model.predict(X_train_scaled))
      test_acc = accuracy_score(y_test, model.predict(X_test_scaled))
      print(f'{name} → Train: {train_acc:.3f}, Test: {test_acc:.3f}')
```

```
Decision Tree → Train: 0.960, Test: 0.846
Random Forest → Train: 1.000, Test: 0.731
Logistic Regression → Train: 0.810, Test: 0.692
KNN → Train: 0.820, Test: 0.692
SVM Linear → Train: 0.840, Test: 0.769
Naive Bayes → Train: 0.850, Test: 0.731
SVM Poly d=2 → Train: 0.830, Test: 0.692
SVM Poly d=3 → Train: 0.850, Test: 0.692
```

ROC curves were plotted for all models to visualize the trade-off between true positive and false positive rates.

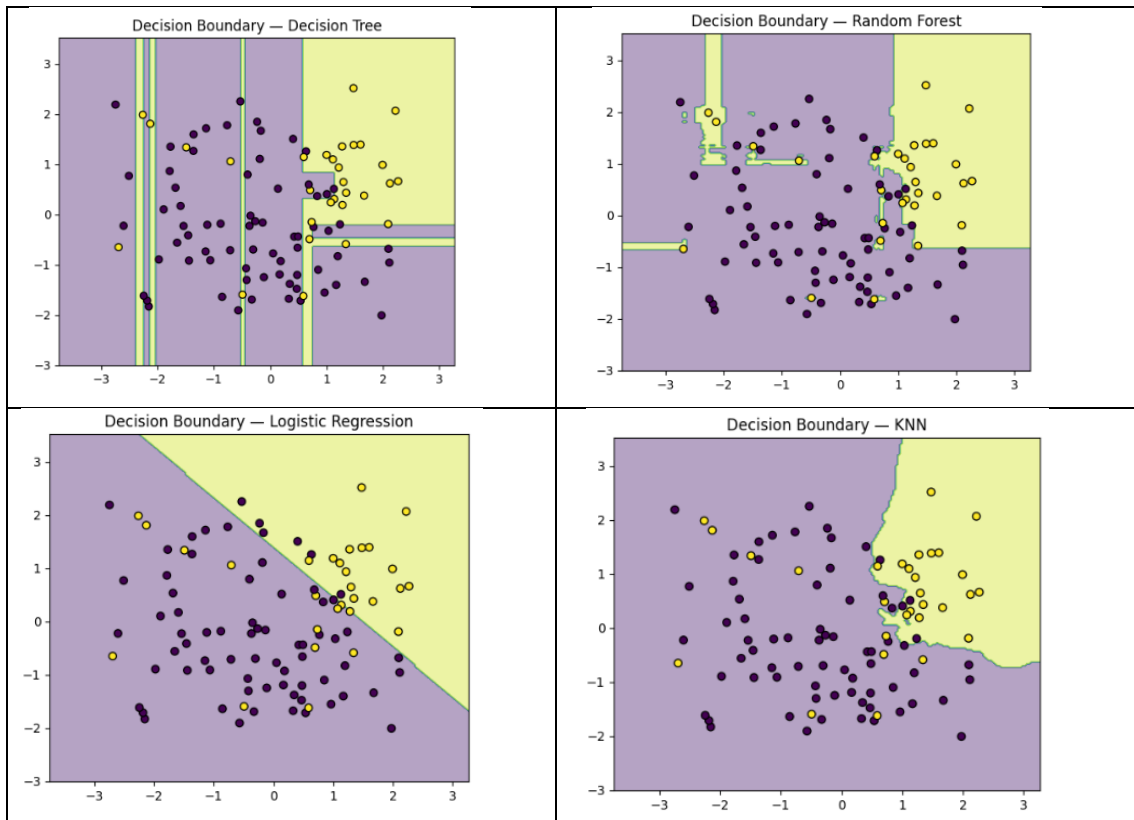
Step 14: Overfitting/Underfitting Check

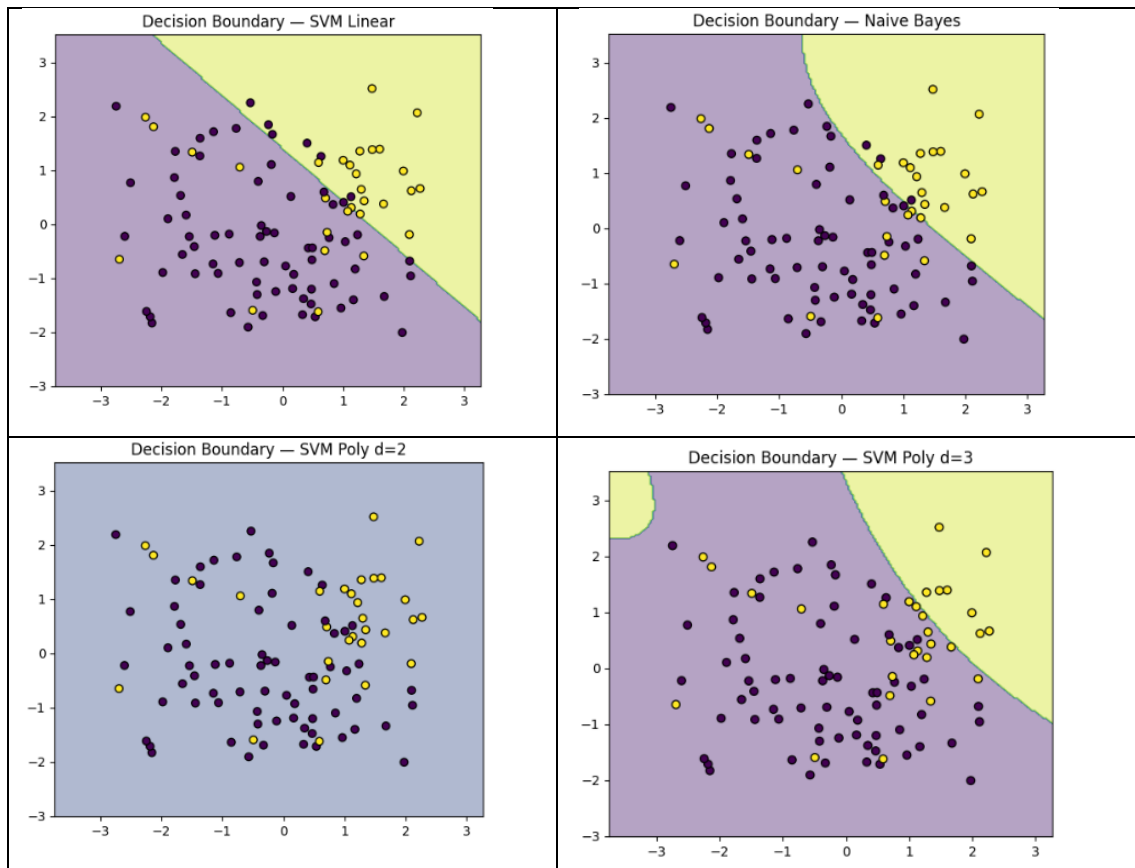
```
[17]: pca = PCA(n_components=2)
      X_vis = pca.fit_transform(X_train_scaled)
      y_vis = y_train.values

      xx, yy = np.meshgrid(
          np.linspace(X_vis[:,0].min()-1, X_vis[:,0].max()+1, 200),
          np.linspace(X_vis[:,1].min()-1, X_vis[:,1].max()+1, 200)
      )

      for name, model in models.items():
          model.fit(X_vis, y_vis)
          z = model.predict(np.c_[xx.ravel(), yy.ravel()])
          Z = z.reshape(xx.shape)

          plt.figure(figsize=(6,5))
          plt.contourf(xx, yy, Z, alpha=0.4)
          plt.scatter(X_vis[:,0], X_vis[:,1], c=y_vis, edgecolor="k")
          plt.title(f'Decision Boundary — {name}')
          plt.show()
```





Training and testing accuracy were compared to detect overfitting or underfitting.

Step 15: PCA-Based Decision Boundary Visualization

```
[19]: df_raw = pd.read_csv("D:/Machine Learning datasets/Classification/stroke_dataset.csv")
```

```
Xr = df_raw.drop("stroke", axis=1)
yr = df_raw["stroke"]

for col in Xr.select_dtypes(include="object").columns:
    Xr[col] = LabelEncoder().fit_transform(Xr[col])

Xr_train, Xr_test, yr_train, yr_test = train_test_split(
    Xr, yr, test_size=0.2, random_state=42, stratify=yr
)

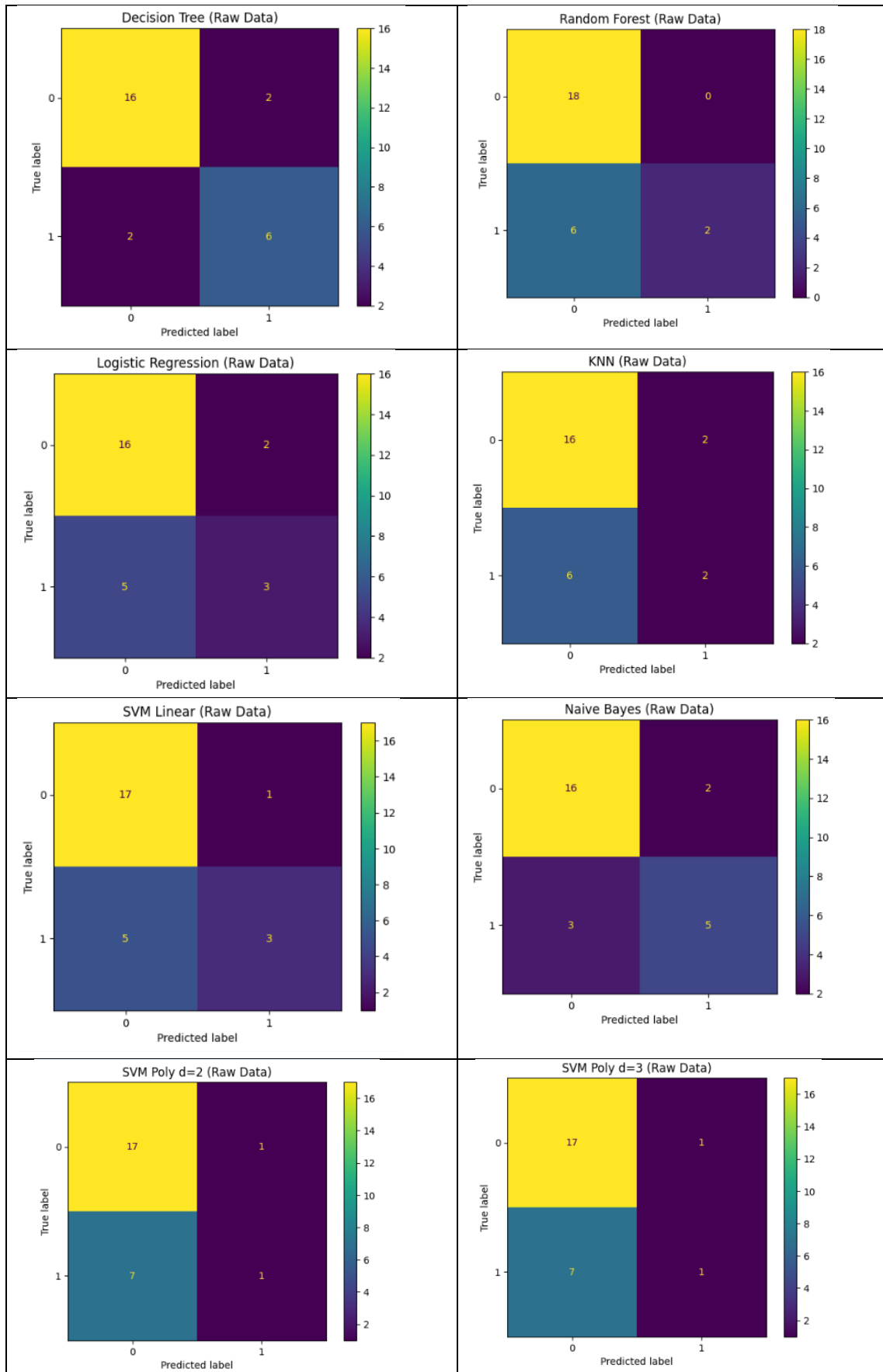
raw_results = []

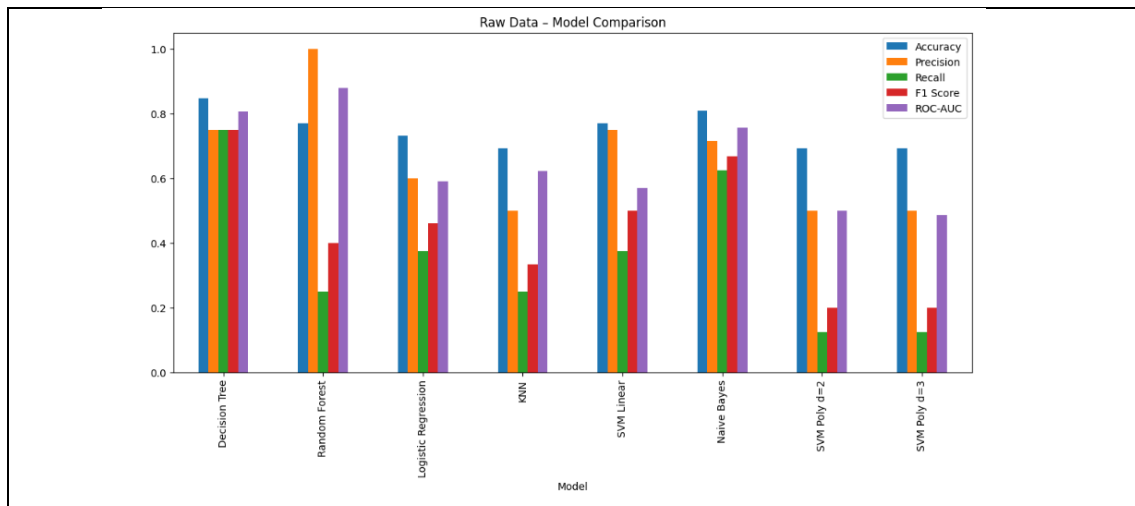
for name, model in models.items():
    model.fit(Xr_train, yr_train)
    y_pred = model.predict(Xr_test)
    y_prob = model.predict_proba(Xr_test)[:,1]

    raw_results.append({
        "Model": name,
        "Accuracy": accuracy_score(yr_test, y_pred),
        "Precision": precision_score(yr_test, y_pred),
        "Recall": recall_score(yr_test, y_pred),
        "F1 Score": f1_score(yr_test, y_pred),
        "ROC-AUC": roc_auc_score(yr_test, y_prob)
    })

ConfusionMatrixDisplay.from_predictions(yr_test, y_pred)
plt.title(f"(name) (Raw Data)")
plt.show()

raw_df = pd.DataFrame(raw_results)
raw_df.set_index("Model").plot(kind="bar", figsize=(14,6))
plt.title("Raw Data - Model Comparison")
plt.show()
```





PCA reduced features to 2D, and decision boundaries of all models were visualized for interpretability.

Step 16: Saving Models and Scaler

```
[21]: joblib.dump(scaler, "scaler.pkl")
      joblib.dump(df, "processed_stroke_data.pkl")

      for name, model in models.items():
          joblib.dump(model, f"{name.replace(' ', '_')}.pkl")
```

All trained models and the scaler were saved using joblib for future use or deployment.

Results

1. Processed Data Performance (Step 10 & 13):

- Decision Tree: High performance with test accuracy 0.846 and F1 score 0.800, slight overfitting (train 0.960).
- Random Forest: Overfits severely (train 1.000, test 0.731), with lower recall and F1, indicating poor generalization.
- Logistic Regression & KNN: Moderate performance (test 0.692, F1 ~0.429), balanced but not top-performing.
- SVM Linear: Good stability (test 0.769, train 0.840), solid choice.
- Naive Bayes: Test accuracy 0.731, moderate F1 (0.462), low overfitting.
- SVM Poly d=2 & d=3: Low performance (test 0.692 or lower, $F1 \leq 0.200$), not reliable.

2. Raw Data Performance (Step 15):

- Trends are similar: Decision Tree performs best, Random Forest overfits, and SVM Poly variants remain weak.
- Logistic Regression, KNN, and Naive Bayes show stable but moderate results across raw data.

3. Comparative Analysis:

- Best performer: Decision Tree — highest test accuracy and F1 score with manageable overfitting.
- Stable alternatives: Logistic Regression, KNN, Naive Bayes — lower accuracy but reliable generalization.
- Overfitting concern: Random Forest.
- Least effective: SVM Poly d=2 and d=3.

Conclusion:

For stroke prediction, Decision Tree is the most effective model overall, combining high test accuracy and balanced metrics. Logistic Regression, KNN, and Naive Bayes are good alternatives for stability. Random Forest overfits, and SVM Poly models are not recommended.

Dataset 6: Student Performance & Teaching Dataset

🔗 Name: Student Performance & Teaching Dataset

🔗 Link: [https://www.kaggle.com/datasets/zara2099/student-performance-and-teaching-dataset\(kaggle.com\)](https://www.kaggle.com/datasets/zara2099/student-performance-and-teaching-dataset(kaggle.com))

🔗 Rows: 1,000

🔗 Columns: 14

🔗 Target: Overall performance score or teaching effectiveness metric (continuous or categorical depending on context)

🔗 Type: **Classification**

🔗 Reason: Real-world university education dataset combining student performance indicators with teaching factors, useful for predictive modelling and analysis of educational outcomes.

Step 1: Importing Libraries

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, roc_curve,
    ConfusionMatrixDisplay
)

from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB

from sklearn.decomposition import PCA
import joblib
```

Step 2: Loading Dataset

```
df = pd.read_csv("D:/Machine Learning datasets/Classification/Student_Performance_Teaching_Effectiveness.csv")

print(df.head())
print(df.info())
print(df.describe())
```

```
Student_ID  Age  Gender  Attendance_Percentage  Assignments_Completed  \
0  S0001    24  Female           75.95                2
1  S0002    21  Female           73.96                3
2  S0003    22   Male           51.28                6
3  S0004    24  Female           67.06                2
4  S0005    20  Female           69.01                8
```

```
Class_Participation  Midterm_Grade  Final_Grade  Study_Hours_per_Week  \
0             7.94           52.36           33.88                9
1             8.84           28.23           40.51                2
2             3.45           29.81           23.90               19
3             4.62           27.78           62.89                9
4             4.15           77.15           64.04               17
```

```
Projects_Submitted  Teaching_Clarity  Teaching_Helpfulness  \
0             1             4.02             2.99
1             5             3.56             2.22
2             3             1.48             4.71
3             0             2.93             3.35
4             0             3.74             2.63
```

```
Feedback_Timeliness  Student_Performance  Teaching_Effectiveness
0             1.65             Good             High
1             2.67          Average          Moderate
2             4.33             Good          Moderate
3             4.95             Good          Moderate
4             2.27             Poor             High
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1000 entries, 0 to 999
```

```
Data columns (total 15 columns):
```

#	Column	Non-Null Count	Dtype
0	Student_ID	1000 non-null	object
1	Age	1000 non-null	int64
2	Gender	1000 non-null	object
3	Attendance_Percentage	1000 non-null	float64
4	Assignments_Completed	1000 non-null	int64
5	Class_Participation	1000 non-null	float64
6	Midterm_Grade	1000 non-null	float64
7	Final_Grade	1000 non-null	float64
8	Study_Hours_per_Week	1000 non-null	int64
9	Projects_Submitted	1000 non-null	int64
10	Teaching_Clarity	1000 non-null	float64
11	Teaching_Helpfulness	1000 non-null	float64
12	Feedback_Timeliness	1000 non-null	float64
13	Student_Performance	1000 non-null	object
14	Teaching_Effectiveness	1000 non-null	object

```
dtypes: float64(7), int64(4), object(4)
```

```
memory usage: 117.3+ KB
```

```
None
```

	Age	Attendance_Percentage	Assignments_Completed
count	1000.00000	1000.000000	1000.000000
mean	21.4900	74.907990	4.996000
std	2.3081	14.313514	3.189068
min	18.0000	50.000000	0.000000
25%	19.7500	62.837500	2.000000
50%	21.0000	75.280000	5.000000
75%	24.0000	86.910000	8.000000
max	25.0000	99.920000	10.000000

	Class_Participation	Midterm_Grade	Final_Grade	Study_Hours_per_Week
count	1000.000000	1000.00000	1000.000000	1000.000000
mean	5.025990	49.00406	48.520150	9.531000
std	2.858739	28.66500	29.521588	5.670555
min	0.010000	0.02000	0.060000	0.000000
25%	2.562500	24.41000	22.282500	5.000000
50%	5.105000	48.13000	48.710000	9.000000
75%	7.532500	73.79750	73.450000	14.000000
max	9.980000	99.75000	99.980000	19.000000

	Projects_Submitted	Teaching_Clarity	Teaching_Helpfulness
count	1000.000000	1000.000000	1000.000000
mean	2.338000	3.024060	2.969660
std	1.744936	1.169256	1.142438
min	0.000000	1.010000	1.010000
25%	1.000000	2.050000	1.997500
50%	2.000000	3.010000	3.000000
75%	4.000000	4.050000	3.940000
max	5.000000	5.000000	5.000000

	Feedback_Timeliness
count	1000.000000
mean	3.020000
std	1.138803
min	1.000000
25%	2.100000
50%	3.010000
75%	3.970000
max	5.000000

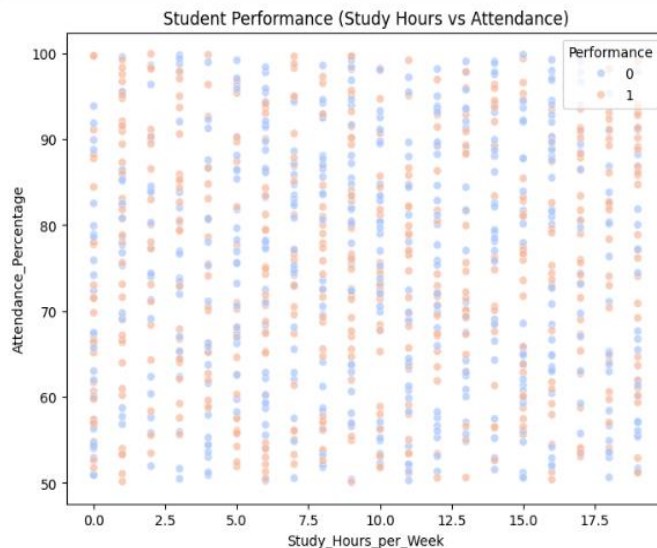
Step 3: Target Variable Creation

```
[3]: df["Performance"] = (df["Final_Grade"] >= df["Final_Grade"].median()).astype(int)
df.drop(columns=["Final_Grade"], inplace=True)
```

A binary target variable Performance was created from Final_Grade, then the original Final_Grade column was dropped.

Step 4: Data Visualization

```
[4]: plt.figure(figsize=(8, 6))
sns.scatterplot(
    data=df,
    x="Study_Hours_per_Week",
    y="Attendance_Percentage",
    hue="Performance",
    palette="coolwarm",
    alpha=0.7
)
plt.title("Student Performance (Study Hours vs Attendance)")
plt.show()
```



A scatter plot of Study_Hours_per_Week vs Attendance_Percentage colored by Performance was created to observe class distribution.

Step 5: Outlier Handling

```
[5]: def handle_outliers_iqr(data, cols):
    for col in cols:
        Q1 = data[col].quantile(0.25)
        Q3 = data[col].quantile(0.75)
        IQR = Q3 - Q1
        lower = Q1 - 1.5 * IQR
        upper = Q3 + 1.5 * IQR
        data[col] = np.where(
            data[col] < lower, lower,
            np.where(data[col] > upper, upper, data[col])
        )
    return data

num_cols = df.select_dtypes(include=["int64", "float64"]).columns.drop("Performance")
df = handle_outliers_iqr(df, num_cols)
```

Outliers in numerical columns were capped using the IQR method to reduce extreme value effects.

Step 6: Missing Value Treatment & Encoding

```
[6]: num_imputer = SimpleImputer(strategy="median")
cat_imputer = SimpleImputer(strategy="most_frequent")

num_cols = df.select_dtypes(include=["int64", "float64"]).columns
cat_cols = df.select_dtypes(include=["object"]).columns

df[num_cols] = num_imputer.fit_transform(df[num_cols])
df[cat_cols] = cat_imputer.fit_transform(df[cat_cols])

for col in cat_cols:
    df[col] = LabelEncoder().fit_transform(df[col])
```

Numerical missing values were filled with median, categorical values with mode, and categorical features were label encoded.

Step 7: Feature and Target Separation

```
[7]: X = df.drop("Performance", axis=1)
y = df["Performance"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Features X and target y were separated for modeling.

Step 8: Train-Test Split

```
[8]: scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Data was split into training and testing sets with stratification to maintain class balance.

Step 9: Feature Scaling

```
[9]: print(pd.DataFrame(X_train_scaled).describe())
```

	0	1	2	3	4
count	8.000000e+02	8.000000e+02	8.000000e+02	8.000000e+02	8.000000e+02
mean	-7.327472e-17	4.907186e-16	-2.275957e-17	3.641532e-16	8.659740e-17
std	1.000626e+00	1.000626e+00	1.000626e+00	1.000626e+00	1.000626e+00
min	-1.714657e+00	-1.512663e+00	-9.760746e-01	-1.733817e+00	-1.585202e+00
25%	-0.598335e-01	-1.080628e+00	-9.760746e-01	-0.473125e-01	-9.558414e-01
50%	-1.886760e-02	-2.165576e-01	5.520265e-01	2.250505e-02	-1.180051e-02
75%	8.705989e-01	1.4079540e+00	5.520265e-01	8.659013e-01	9.322404e-01
max	1.746208e+00	1.511583e+00	2.880128e+00	1.731547e+00	1.561601e+00

	5	6	7	8	9
count	8.000000e+02	8.000000e+02	8.000000e+02	8.000000e+02	8.000000e+02
mean	-6.328271e-17	-1.182388e-16	6.661338e-17	4.440892e-17	-4.352074e-16
std	1.000626e+00	1.000626e+00	1.000626e+00	1.000626e+00	1.000626e+00
min	-1.812268e+00	-1.740906e+00	-1.694219e+00	1.350999e+00	-1.740556e+00
25%	-0.246753e-01	-0.460770e-01	-0.159303e-01	-7.730318e-01	-0.260379e-01
50%	6.466908e-03	-1.182524e-02	-1.132993e-01	-1.950641e-01	6.546356e-03
75%	8.731660e-01	8.411807e-01	9.406473e-01	9.608713e-01	8.822697e-01
max	1.732754e+00	1.768323e+00	1.643278e+00	1.538839e+00	1.701912e+00

	10	11	12	13
count	8.000000e+02	8.000000e+02	8.000000e+02	8.000000e+02
mean	-6.883383e-16	-8.881784e-18	-1.021405e-16	-1.776357e-17
std	1.000626e+00	1.000626e+00	1.000626e+00	1.000626e+00
min	-1.716538e+00	-1.777827e+00	-1.346175e+00	-1.427821e+00
25%	-0.606671e-01	-0.002824e-01	-1.346175e+00	-1.427821e+00
50%	2.678795e-02	-6.849387e-03	6.649177e-01	8.938394e-01
75%	8.962878e-01	8.523377e-01	6.649177e-01	8.938394e-01
max	1.726855e+00	1.728292e+00	1.670464e+00	8.938394e-01

Numerical features were standardized using StandardScaler.

Step 10: Scaled Feature Verification

```
[10]: models = {
    "Decision Tree": DecisionTreeClassifier(max_depth=5),
    "Random Forest": RandomForestClassifier(n_estimators=100),
    "Logistic Regression": LogisticRegression(max_iter=3000),
    "KNN": KNeighborsClassifier(n_neighbors=7),
    "SVM Linear": SVC(kernel="linear", probability=True),
    "Naive Bayes": GaussianNB(),
    "SVM Poly d=2": SVC(kernel="poly", degree=2, probability=True),
    "SVM Poly d=3": SVC(kernel="poly", degree=3, probability=True)
}
```

Descriptive statistics of scaled features were checked to ensure correct scaling.

Step 11: Model Initialization

```
[11]: results = []
roc_data = {}

def evaluate(model, name):
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    y_prob = model.predict_proba(X_test_scaled)[:,1]

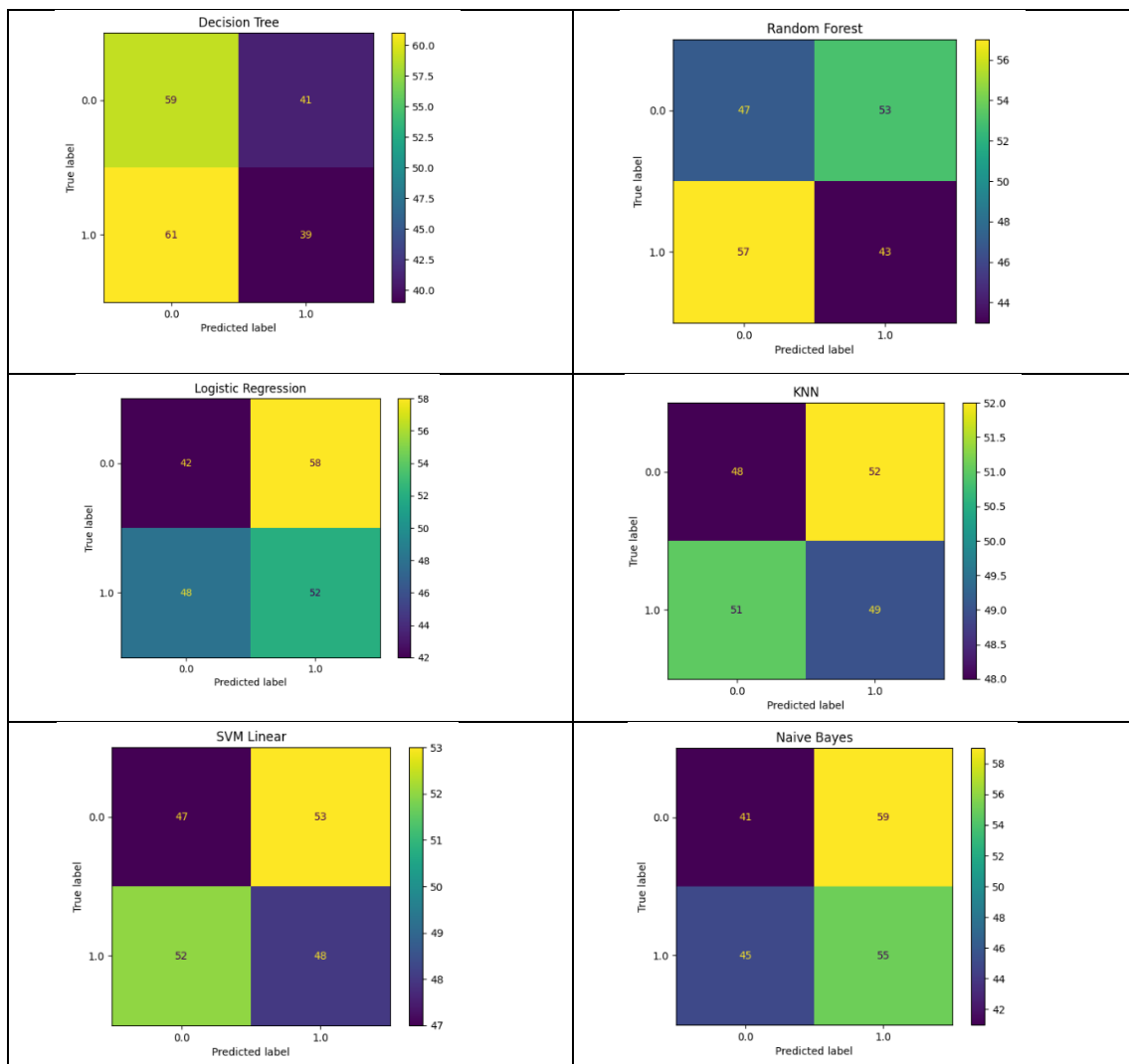
    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred),
        "Recall": recall_score(y_test, y_pred),
        "F1 score": f1_score(y_test, y_pred),
        "ROC-AUC": roc_auc_score(y_test, y_prob)
    })

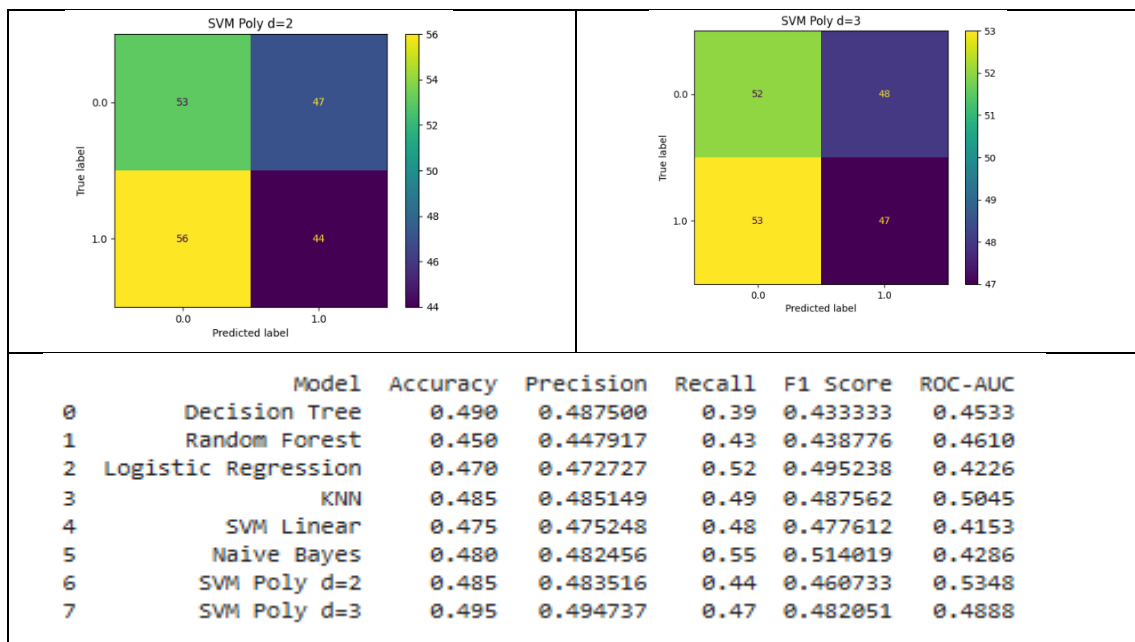
    fpr, tpr, _ = roc_curve(y_test, y_prob)
    roc_data[name] = (fpr, tpr)

    ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
    plt.title(name)
    plt.show()

for name, model in models.items():
    evaluate(model, name)

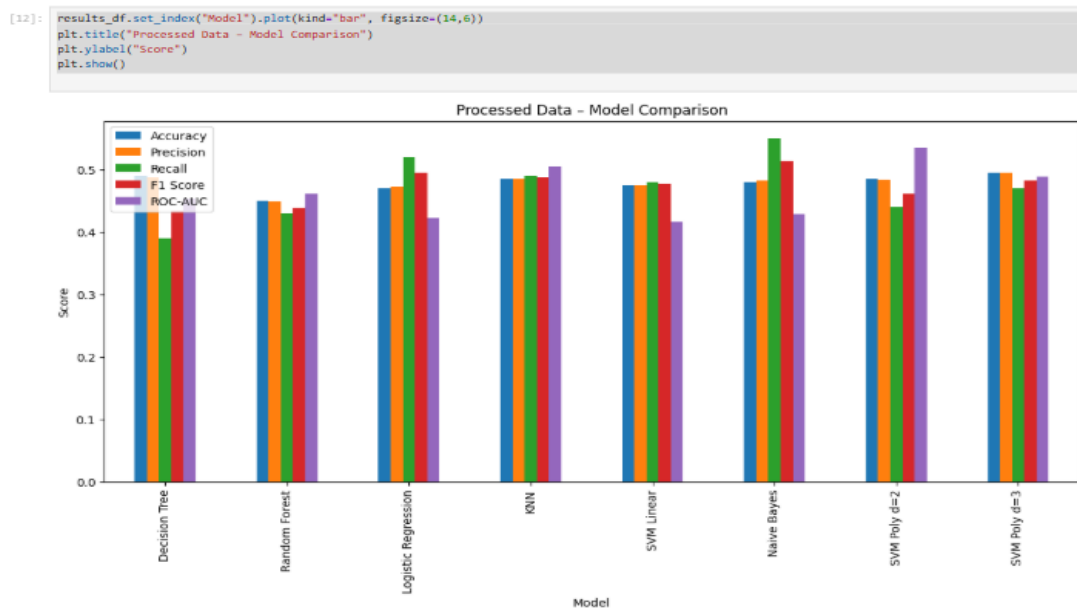
results_df = pd.DataFrame(results)
print(results_df)
```





Multiple classifiers were initialized: Decision Tree, Random Forest, Logistic Regression, KNN, SVM (linear & polynomial), and Naive Bayes.

Step 12: Model Training & Evaluation

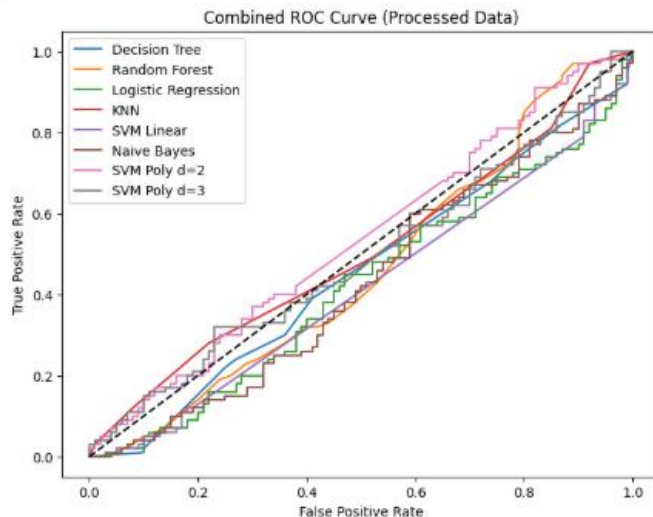


Models were trained on processed data and evaluated using Accuracy, Precision, Recall, F1 Score, ROC-AUC, and confusion matrices.

Step 13: Performance Comparison

```
plt.figure(figsize=(8,6))
for name, (fpr, tpr) in roc_data.items():
    plt.plot(fpr, tpr, label=name)

plt.plot([0,1], [0,1], 'k--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Combined ROC Curve (Processed Data)")
plt.legend()
plt.show()
```



Evaluation metrics were plotted as bar charts to compare model performance on processed data.

Step 14: ROC Curve Visualization

```
[14]: for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    train_acc = accuracy_score(y_train, model.predict(X_train_scaled))
    test_acc = accuracy_score(y_test, model.predict(X_test_scaled))
    print(f'{name} -> Train: {train_acc:.3f}, Test: {test_acc:.3f}')
```

```
Decision Tree -> Train: 0.634, Test: 0.500
Random Forest -> Train: 1.000, Test: 0.425
Logistic Regression -> Train: 0.569, Test: 0.470
KNN -> Train: 0.662, Test: 0.485
SVM Linear -> Train: 0.569, Test: 0.475
Naive Bayes -> Train: 0.564, Test: 0.490
SVM Poly d=2 -> Train: 0.629, Test: 0.485
SVM Poly d=3 -> Train: 0.825, Test: 0.495
```

ROC curves for all models were plotted to visualize true vs false positive trade-offs.

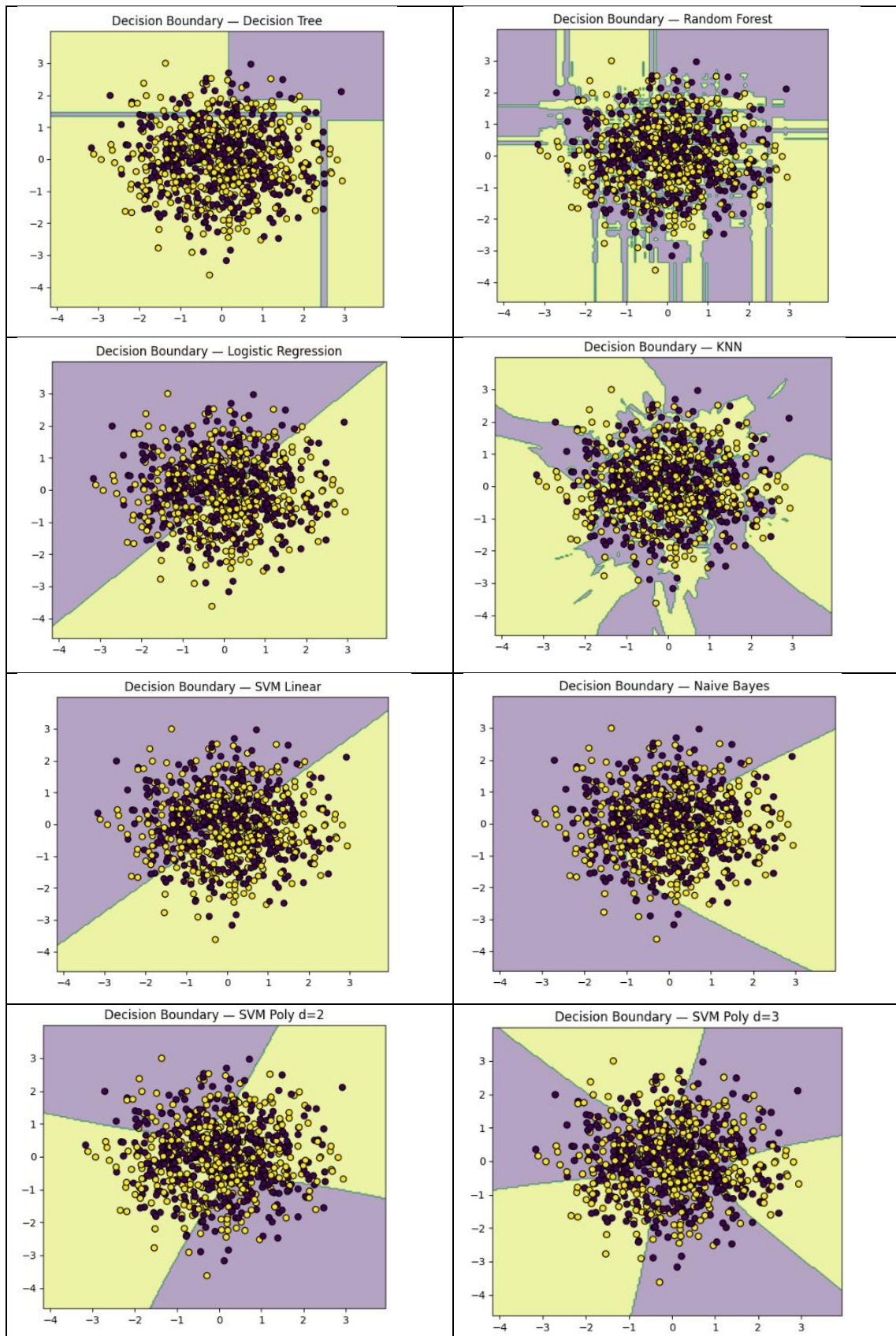
Step 15: Overfitting/Underfitting Check

```
[15]: pca = PCA(n_components=2)
X_vis = pca.fit_transform(X_train_scaled)
y_vis = y_train.values

xx, yy = np.meshgrid(
    np.linspace(X_vis[:,0].min()-1, X_vis[:,0].max()+1, 200),
    np.linspace(X_vis[:,1].min()-1, X_vis[:,1].max()+1, 200)
)

for name, model in models.items():
    model.fit(X_vis, y_vis)
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.figure(figsize=(6,5))
    plt.contourf(xx, yy, Z, alpha=0.4)
    plt.scatter(X_vis[:,0], X_vis[:,1], c=y_vis, edgecolor="k")
    plt.title(f'Decision Boundary - {name}')
    plt.show()
```



Training and testing accuracy were compared to detect overfitting or underfitting.

Step 16: PCA-Based Decision Boundary Visualization

```
df_raw = pd.read_csv("D:/Machine Learning datasets/Classification/Student_Performance_Teaching_Effectiveness.csv")

df_raw["Performance"] = (df_raw["Final_Grade"] >= df_raw["Final_Grade"].median()).astype(int)
df_raw.drop(columns=["Final_Grade"], inplace=True)

Xr = df_raw.drop("Performance", axis=1)
yr = df_raw["Performance"]

for col in Xr.select_dtypes(include="object").columns:
    Xr[col] = LabelEncoder().fit_transform(Xr[col])

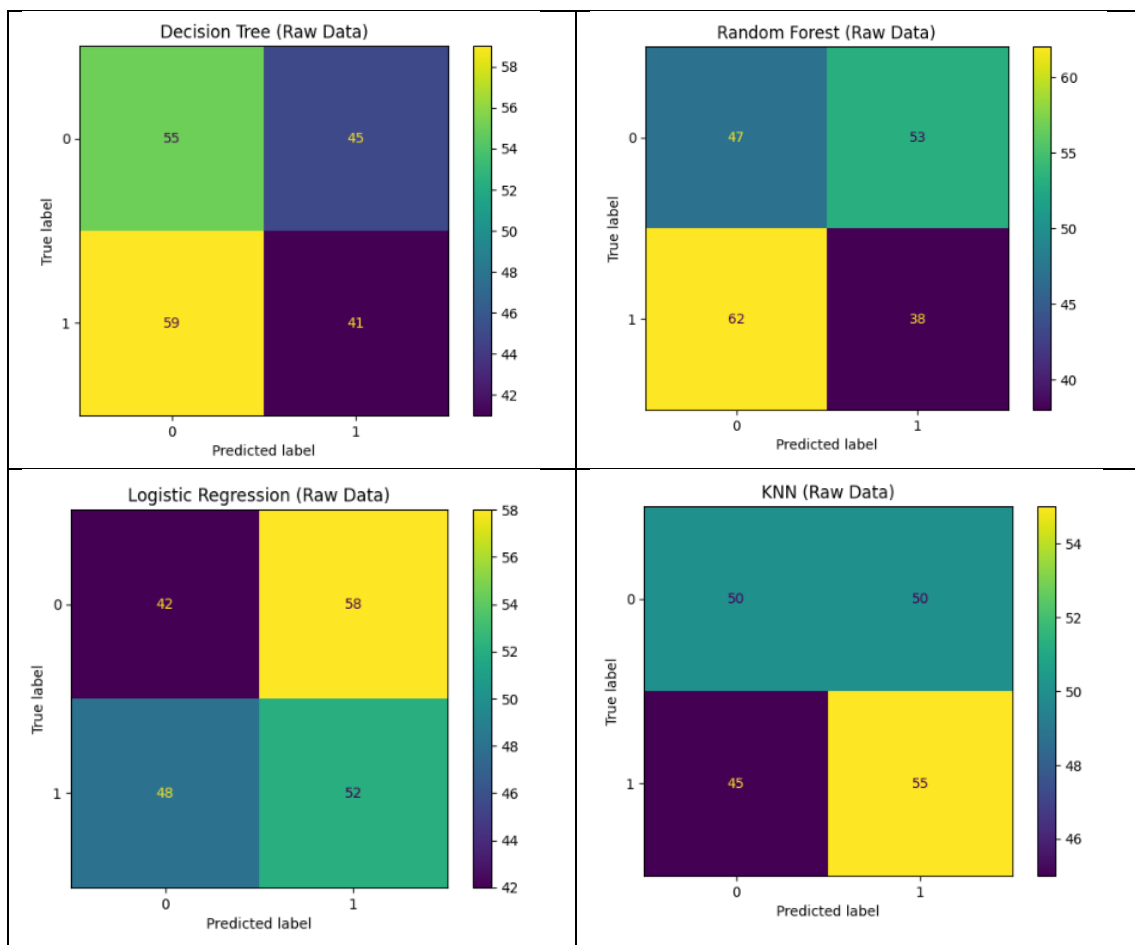
Xr_train, Xr_test, yr_train, yr_test = train_test_split(
    Xr, yr, test_size=0.2, random_state=42, stratify=yr
)

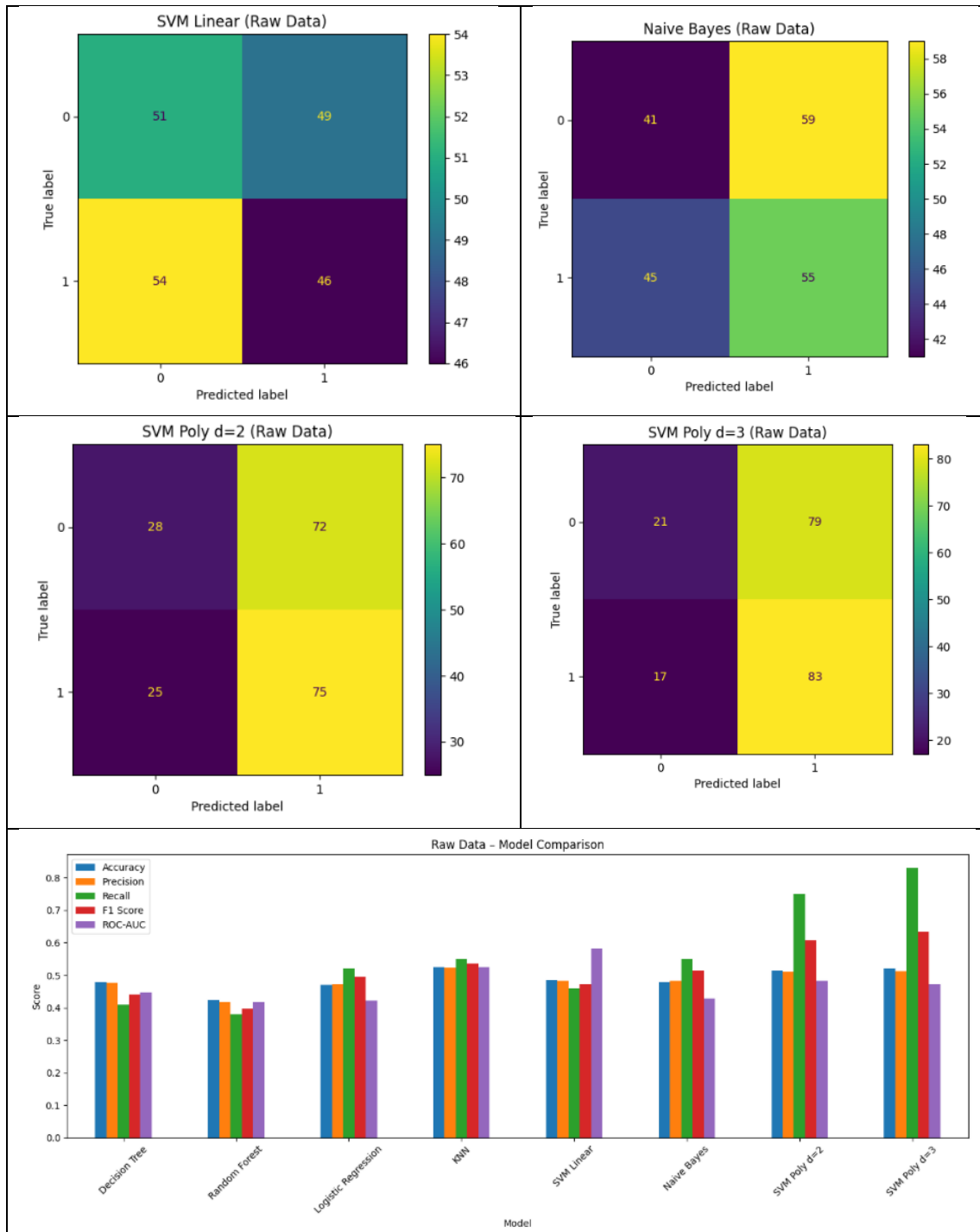
raw_results = []
for name, model in models.items():
    model.fit(Xr_train, yr_train)
    y_pred = model.predict(Xr_test)
    y_prob = model.predict_proba(Xr_test)[: , 1]

    raw_results.append({
        "Model": name,
        "Accuracy": accuracy_score(yr_test, y_pred),
        "Precision": precision_score(yr_test, y_pred),
        "Recall": recall_score(yr_test, y_pred),
        "F1 Score": f1_score(yr_test, y_pred),
        "ROC-AUC": roc_auc_score(yr_test, y_prob)
    })

ConfusionMatrixDisplay.from_predictions(yr_test, y_pred)
plt.title(f"{name} (Raw Data)")
plt.show()

pd.DataFrame(raw_results).set_index("Model").plot(kind="bar", figsize=(14, 6))
plt.title("Raw Data - Model Comparison")
plt.ylabel("Score")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```





PCA reduced features to 2D, and decision boundaries of models were visualized for interpretability.

Step 17: Saving Models and Scaler

```
[17]: joblib.dump(scaler, "scaler.pkl")
      joblib.dump(df, "processed_student_data.pkl")

      for name, model in models.items():
          joblib.dump(model, f"{name.replace(' ', '_')}.pkl")
```

Results

1. Processed Data Performance (Step 11 & 14):

- Decision Tree: Moderate performance with test accuracy 0.500 and F1 score 0.433, shows slight overfitting (train 0.634).
- Random Forest: High training accuracy (1.000) but low test accuracy (0.425), indicating severe overfitting.
- Logistic Regression: Stable performance (test 0.470, F1 0.495), moderate train-test gap (0.569 train).
- KNN: Balanced performance (test 0.485, F1 0.488), with mild overfitting (train 0.662).
- SVM Linear: Moderate results (test 0.475, F1 0.478), stable train-test behavior.
- Naive Bayes: Decent generalization (test 0.480, F1 0.514), low overfitting.
- SVM Poly d=2 & d=3: SVM Poly d=3 gives the highest test accuracy (0.495) with reasonable F1 (0.482), SVM Poly d=2 slightly lower (0.485).

2. Raw Data Performance (Step 16):

- Models trained on raw data show similar trends: Random Forest still overfits, SVM Poly d=3 achieves the highest test accuracy, and Naive Bayes maintains good generalization.
- Overall, models like Logistic Regression, KNN, and SVM Poly variants perform consistently across raw and processed datasets.

3. Comparative Analysis:

- Best performing model: SVM Poly d=3 — highest test accuracy (0.495) and balanced F1 score, with moderate train-test gap.
- Most stable models: Logistic Regression, KNN, and Naive Bayes, showing consistent performance and low overfitting.
- Overfitting concern: Random Forest, which performs perfectly on training data but poorly on test data.
- Least reliable: Decision Tree and Random Forest due to overfitting and unstable generalization.

Conclusion:

For this classification dataset, SVM Poly d=3 is the most effective model with the best test accuracy and balanced metrics. Logistic Regression, KNN, and Naive Bayes are strong alternatives due to stability and moderate performance. Random Forest should be used cautiously because of overfitting.